

Deep Learning for NLP

Student name: *Margarita Orfanidi*

ID: *7115152400023*

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*

Semester: *Spring Semester 2025-2026*

Contents

1	Abstract	3
2	Analysis	3
2.1	Dataset	3
2.2	Exploratory Data Analysis	4
2.2.1	Samples	4
2.2.2	Descriptive Analysis	5
2.2.3	Textual Analysis	10
2.2.4	Token Analysis	13
3	Pre-processing	15
4	Models	17
4.1	Prompting Techniques	18
4.1.1	Zero-shot Prompting	18
4.1.2	Few-shot Prompting	19
4.1.3	Chain-of-Thought Prompting	20
4.1.4	Chain-of-Thought with Few-shot Prompting	21
4.1.5	Self-Consistency	21
5	Evaluation Metrics	21
6	Experiments	23
6.1	Implementation Details	23
6.2	Output Parsing and Label Mapping	26
6.3	Experimental Results	27
6.3.1	Qwen3.5-0.8B Experiments	27
6.3.2	Qwen3.5-2B Experiments	39
6.3.3	Gemma 3 1B	45
6.3.4	Gemma 3 4B	47
6.4	Overall Results	48
7	Error Analysis	53

8	Conclusion and model selection	56
9	Comparison with the other projects	56

1. Abstract

This assignment investigates ambiguity detection in politicians' answers during interviews, using the SemEval 2026 Task 6 (CLARITY) dataset. The task is framed as a three-class classification problem, where each question-answer pair is classified as Clear Reply, Ambivalent, or Clear Non-Reply.

The study evaluates pretrained instruction-tuned large language models using prompting-based methods. It compares different prompting strategies as well as models of different sizes.

2. Analysis

2.1. Dataset

The dataset consists of a collection of questions and answers from Presidents of the United States during interviews. The dataset is divided into two parts: a training set and a test set. The training set contains 3,448 entries with 20 attributes, while the test set contains 308 entries with the same attributes. The columns are described in the following table.

Column	Description
title	Interview title
date	Date of the interview
president	President being interviewed
url	Source URL of the interview
question_order	Order of the question in the interview
interview_question	Original interview question
interview_answer	Answer given by the president
gpt3.5_summary	Summary generated by GPT-3.5
gpt3.5_prediction	Prediction generated by GPT-3.5
question	Sub question
annotator_id	Identifier of the annotator
annotator1	Label from annotator 1
annotator2	Label from annotator 2
annotator3	Label from annotator 3
inaudible	Indicates if the segment contains inaudible parts
multiple_questions	Indicates if the question contains multiple parts
affirmative_questions	Indicates if the question is affirmative
index	Entry index
clarity_label	Label indicating response clarity
evasion_label	Label indicating response evasion

Table 1: Columns of the SemEval 2026 Task 6 dataset.

The dataset includes textual components such as the original interview question, the sub-questions, and the corresponding interview response. Moreover, it contains metadata, such as the interview title, date, and the name of the president. Finally, the dataset includes the proposed classification labels provided at the sub-question level,

with each sub-question assigned classification labels (clarity_label and evasion_label) based on the proposed taxonomy.

It is important to note that the training set does not contain the individual labels assigned by each annotator, but only the final aggregated labels. Therefore, although the dataset generally contains non-null values, all values in these specific columns are null.

In contrast, in the test set, most metadata fields are null, while the columns necessary for evaluation are not null, such as the main text columns, the individual annotator labels (annotator1, annotator2, annotator3), and the corresponding labels.

2.2. Exploratory Data Analysis

Before developing the classification models, a study of the data and the nature of the problem must first be conducted. It is necessary to examine basic statistics, understand the distribution of the labels, and identify potential patterns and correlations in the data. Through this process, we can confirm the structure of the dataset, recognize the most important variables, and develop a clearer understanding of the problem we are trying to solve.

2.2.1. Samples. Random samples were displayed from the dataset in order to observe the structure and formatting of the textual data. This qualitative inspection helps identify potential patterns and understand how the text appears in practice, allowing us to draw conclusions about the preprocessing steps that may be required. For example, in the dataset, several spelling inconsistencies, missing spaces, and the presence of punctuation marks were observed, creating an initial intuition about the type of preprocessing that may be needed.

Sample 2900	
	Value
title	The President's News Conference With President Lee Myung-bak of South Korea at Camp David, Maryland
date	April 19, 2008
president	George W. Bush
url	https://www.presidency.ucsb.edu/documents/the-presidents-news-conference-with-president-lee-myung-bak-south-korea-camp-david
question_order	2
interview_question	Q. I have a question for President Lee. Korea and the United States have made many achievements through the summit meeting, especially North Korean nuclear issue and the strengthening of the alliance. As for North Korean nuclear issue, Mr. President Lee suggested setting up a permanent liaison office in both Seoul and Pyongyang. What are some of the follow-up effects, if you do have any follow-up actions? And do you have any thoughts of proposing a meeting with Chairman Kim at an earlier date?
interview_answer	The process is not something that we discussed between ourselves during the summit meeting. In fact, when I was staying in Washington, DC, I had an interview with one of the newspapers there, and it came up. Of course, it was not a sudden suggestion. I did have a meeting among my staff and related ministries, and I talked about this in detail before I came to the United States. We have a new administration in Korea, and we haven't yet begun dialogue with the rs. Inter-Korean dialogue: there is a need for us to have dialogue all the time. Up until now, we had dialogues whenever the need arose, and then it would stop. However, dialogue should be based on genuine cooperation and sincerity. And so with this in mind, I thought that it would be helpful to set up a permanent liaison office in both Seoul and Pyongyang. As for the summit meeting between myself and Chairman Kim, I will agree to it when the need is real. And I already said publicly that I am willing to meet with him—not just once, but many times—but if the meeting will yield substantial and real results, I believe only when that is possible, I am ready to meet with him and have sincere dialogue, because that will help to bring about peace and stability of the peninsula. So basically, I do hold that thought, but I'm not suggesting that—to have a meeting with Chairman Kim anytime soon, but I'm ready to meet with him. Steven Lee [Steven Lee Myers, New York Times].
gpt3.5_summary	The question consists of 2 parts:\n1. Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue.\n2. Thoughts on proposing a meeting with Chairman Kim at an earlier date.\n<- Summary Answer ->\n\nThe response provides the following information regarding these points:\n1. Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue:\n- President Lee suggests setting up a permanent liaison office in both Seoul and Pyongyang to facilitate dialogue and cooperation between North Korea and South Korea.\n- The suggestion was made during an interview with a newspaper in Washington, DC, and was not discussed during the summit meeting.\n- The intention behind the suggestion is to establish genuine cooperation and sincerity in inter-Korean dialogue.\n2. Thoughts on proposing a meeting with Chairman Kim at an earlier date:\n- President Lee is open to the idea of a meeting with Chairman Kim, but only when the need is real and there is a possibility of substantial and real results.\n- He believes that sincere dialogue with Chairman Kim can contribute to the peace and stability of the Korean peninsula.\n- While he holds the thought of meeting with Chairman Kim, he is not suggesting a meeting anytime soon, but is willing to meet if the need arises.\n\nOverall, President Lee emphasizes the importance of genuine cooperation, sincerity, and substantial results in both the setup of a liaison office and the possibility of a meeting with Chairman Kim.
gpt3.5_prediction	Question 1: Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue.\nVerdict: 1.1 Explicit - The information requested is explicitly stated (in the requested form).\nExplanation: The response explicitly provides information about President Lee's suggestion to set up a permanent liaison office in Seoul and Pyongyang. It explains that the suggestion was made to facilitate dialogue and cooperation between North and South Korea and highlights the intention behind the suggestion.\nQuestion 2: Thoughts on proposing a meeting with Chairman Kim at an earlier date.\nVerdict: 2.4 General - The information provided is too general/lacks the requested specificity.\nExplanation: The response does not specifically address the question of proposing a meeting with Chairman Kim at an earlier date. While President Lee expresses willingness to meet with Chairman Kim when the need is real and there can be substantial results, he does not provide a direct response to the request for thoughts on proposing an earlier meeting. The answer is more general in nature.
question	Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue.
annotator_id	85
annotator1	None
annotator2	None
annotator3	None
inaudible	False
multiple_questions	False
affirmative_questions	True
index	2900
clarity_label	Ambivalent
evasion_label	Implicit

Figure 1: Sample

In addition, redundancy in the interview responses was observed. This occurs because some responses are similar despite referring to different questions, but mainly because the responses are labeled based on whether they address each sub-question.

As a result, the same response may appear multiple times in the dataset, depending on the number of sub-questions associated with a particular interview question. This behavior is illustrated below by presenting the original question, the corresponding sub-questions, the interview response, and the assigned label. As shown, the same response is repeated across different sub-questions, while the assigned label may vary depending on the extent to which the response addresses each sub-question.

INTERVIEW QUESTION:

Q. And if I could, on the election, sir--can I--TikTok/Microsoft Corporation Q. President Trump, on the sale of TikTok, you're basically arguing that the U.S. Government is going to collect a cut from a--of a transaction including two companies, in which it doesn't hold a stake in. That's unprecedented. That's never happened in U.S. history before, and the administration has offered very little explanation about how that's going to work. Can you back your statement up and provide specifics about how that would work?

--- Example ---

Sub-question:

The interviewer's request for President Trump to back up his statement and provide specific details on how the government would collect this cut.

Answer:

Did you say, That's impressive? Did you actually use that term?...

Clarity label: Ambivalent

--- Example ---

Sub-question:

The previous comment made by the interviewer, expressing that President Trump is arguing for the U.S. Government to collect a cut from the sale of TikTok.

Answer:

Did you say, That's impressive? Did you actually use that term?...

Clarity label: Ambivalent

Figure 2: Sample

This redundancy can make class separation more difficult, since very similar or even identical answers may receive different labels depending on the specific sub-question being evaluated. As a result, relying only on the response text may not be sufficient, because the same linguistic content does not always imply the same degree of clarity.

2.2.2. Descriptive Analysis.

Clarity Labels Distribution The labels that were used for the classification task are the clarity labels, which include Clear Reply, Clear Non-Reply, and Ambivalent. The distribution of instances across these labels is shown in Figure 3. The majority of instances belong to the Ambivalent class, accounting for approximately 59.2% of the data (2039 samples). The Clear Reply category represents about 30.5% of the dataset (1051 samples), while Clear Non-Reply is the least frequent class with 10.3% (356 samples).

This distribution indicates a noticeable class imbalance, with the Ambivalent class dominating the dataset, a factor that should be considered. Therefore, this imbalance should be considered during evaluation, and additional metrics such as macro-F1 are useful, since they give equal importance to all classes regardless of their frequency.

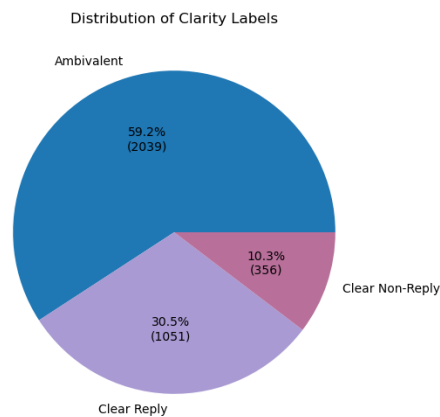


Figure 3: Distribution of clarity labels in the dataset

Evasion Labels Distribution The dataset also includes an `evasion_label`, which is a second, more detailed taxonomy that specifies the type of evasive strategy used in a response.

Figure 4 presents the relationship between clarity labels and evasion labels in the dataset. The evasion categories are Claims ignorance, Clarification, Declining to answer, Deflection, Dodging, Explicit, General, Implicit, and Partial/Half-answer. We can see that Deflection, Dodging, Implicit, General, and Partial/Half-answer correspond to Ambivalent responses, while Claims ignorance, Clarification, and Declining to answer correspond to Clear Non-Reply. The Explicit category corresponds to Clear Reply, meaning that the response is direct and clear.

evasion_label	Claims ignorance	Clarification	Declining to answer	Deflection	Dodging	Explicit	General	Implicit	Partial/half-answer
clarity_label									
Ambivalent	0	0	0	381	706	0	386	488	79
Clear Non-Reply	119	92	145	0	0	0	0	0	0
Clear Reply	0	0	0	0	0	1052	0	0	0

Figure 4: Relationship between clarity labels and evasion labels in the dataset.

Moreover, Figure 5 presents the distribution of the evasion labels in the dataset. The most frequent category is Explicit, accounting for 30.5% of the instances (1051 samples), which corresponds to clear and direct responses. Among the evasive strategies, Dodging appears most frequently (20.5%, 705 samples), followed by Implicit (14.2%, 487 samples), General (11.2%, 386 samples), and Deflection (11.0%, 380 samples). Less common categories include Declining to answer (4.2%), Claims ignorance (3.5%), Clarification (2.7%), and Partial/Half-answer (2.3%).

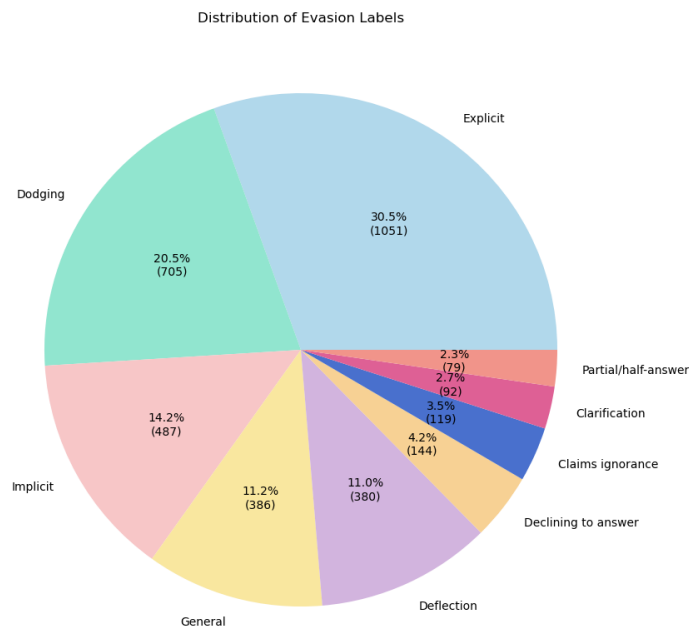
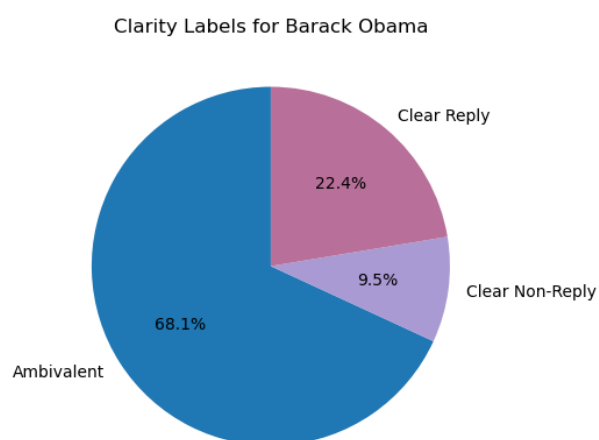
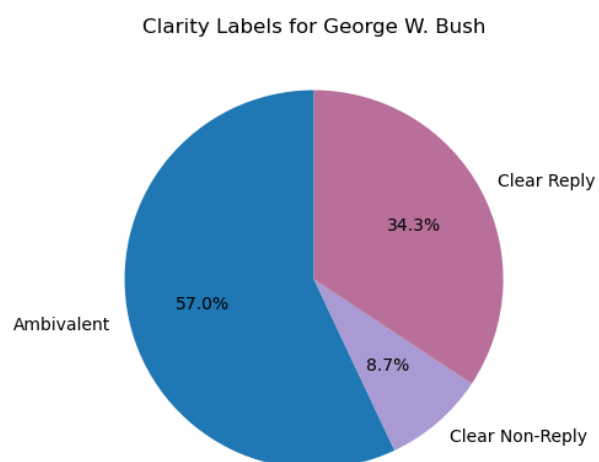
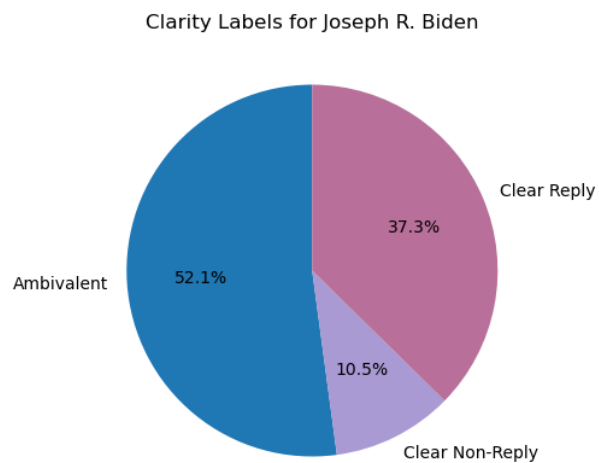
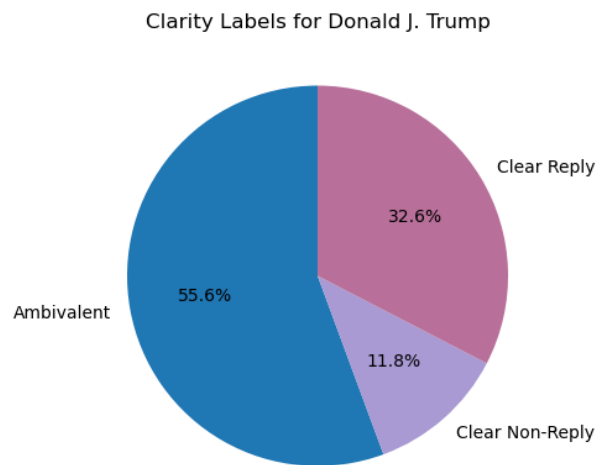


Figure 5: Distribution of evasion labels in the dataset.

Clarity Label Distribution by President It is important to determine whether the nature of the responses in the dataset is speaker-dependent. An analysis of the distribution of clarity labels for each president helps reveal whether certain categories tend to appear more frequently depending on the president who provides the response, rather than representing a general characteristic of the dataset as a whole. As illustrated in the figures, Ambivalent responses constitute the majority for all presidents. However, some differences can still be observed. In particular, Barack Obama exhibits the highest proportion of Ambivalent responses (68.1%), indicating a larger share of indirect or partially evasive answers. In contrast, Donald J. Trump shows a higher proportion of Clear Reply responses (32.6%), suggesting that he provides more direct answers compared to the other presidents.





To further compare these distributions, the label frequencies across presidents are visualized using a grouped bar chart. As shown in the figure, although minor differences can be observed, no substantial variations appear in the overall response patterns

among the presidents. In all cases, Ambivalent responses remain the dominant category, followed by Clear Reply, while Clear Non-Reply appears less frequently. This suggests that the general response behavior is independent across the different presidents in the dataset.

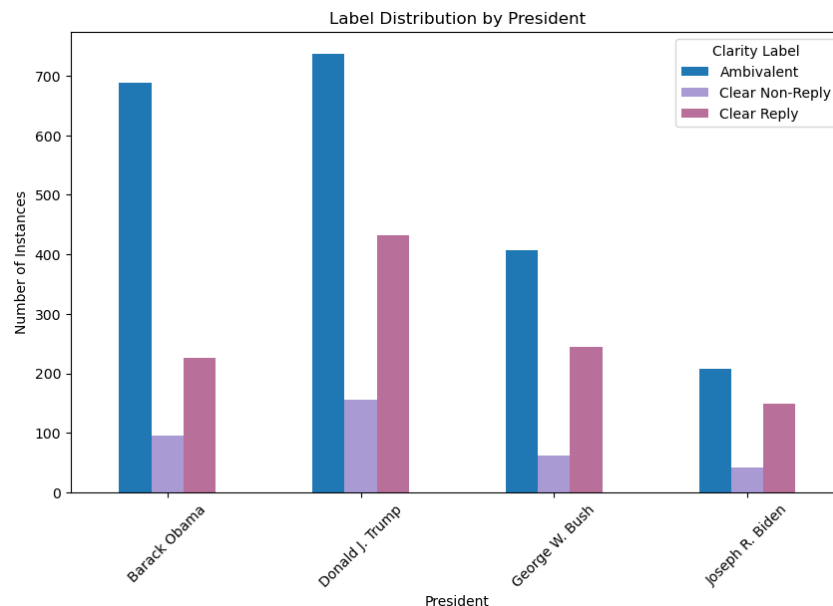


Figure 6: Label distribution per president

Clarity Labels Distribution by Affirmative questions Affirmative questions seem to be associated with slightly more direct responses. In these cases, Ambivalent responses become less frequent (from 60.5% to 54.5%), while Clear Reply (from 29.9% to 32.6%) and Clear Non-Reply responses (from 9.6% to 12.8%) become slightly more common. This suggests that affirmative questions may encourage clearer answers, although the effect remains limited.

clarity_label	Ambivalent	Clear Non-Reply	Clear Reply
affirmative_questions			
False	0.605007	0.096039	0.298954
True	0.545337	0.128238	0.326425

Figure 7: Label distribution per affirmative questions

Clarity Labels Distribution by Year Figure 8 shows a heatmap of interview questions by president and year. The distribution of questions matches the years of each presidency, with questions concentrated in the relevant time periods. Some differences can also be seen in the number of questions per year, since some years contain more instances than others.

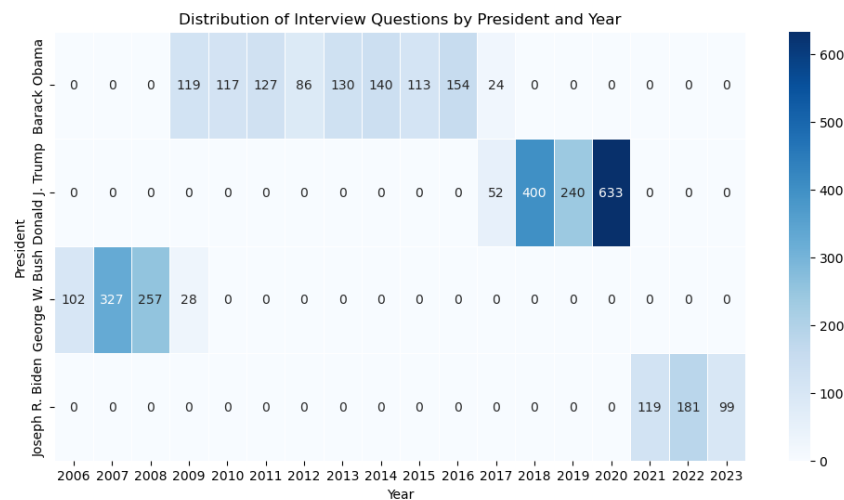


Figure 8: Distribution of interview answers per presidents and year

Figure 9 illustrates the distribution of clarity labels across different years. As observed, there is no significant temporal variation in the distribution, as the relative proportions of the labels remain stable over time.

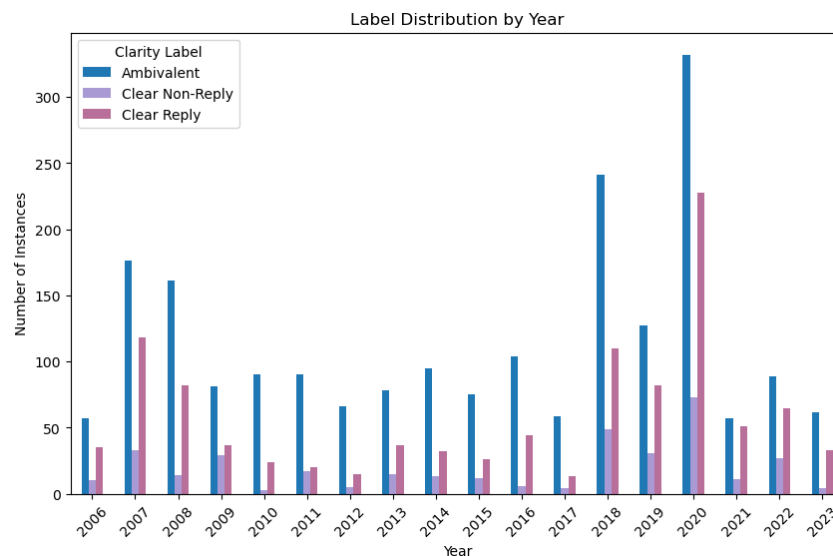


Figure 9: Distribution of clarity labels by year

2.2.3. Textual Analysis. The analysis then focused on the textual characteristics of the dataset. This step helps provide a better understanding of the language used in the interview answers, including aspects such as answer length, vocabulary, and recurring word patterns. It also helps identify whether simple textual properties differ across clarity labels or whether the classes share similar linguistic characteristics.

This is important for the current task because ambiguity is not always expressed through clear lexical cues. Similar vocabulary may appear across different classes, and the same answer may be interpreted differently depending on the question. Therefore, this analysis helps explain why the task is challenging and provides useful context for the design and evaluation of the prompting experiments.

Clarity Label Distribution by Word Count The analysis of answer length, measured in number of words, reveals notable differences across the clarity labels. The dataset contains 3,448 responses, with an average length of approximately 294 words and a high standard deviation (301.54), which indicates a variability in response length. The distribution is highly skewed, ranging from very short answers (1 word) to very long ones (up to 2,117 words). The median value is lower than the mean, suggesting the presence of long outlier responses that increase the average,.

When examining the classes separately, Ambivalent responses not only constitute the majority of the dataset but also tend to be longer on average (331.86 words). Clear Reply responses include 1,052 samples with a moderate average length of 272.04 words. In contrast, Clear Non-Reply responses form the smallest group and are considerably shorter, with an average length of 137.81 words.

Figure 10 illustrates these differences in answer length across the clarity labels. The boxplot shows that responses labeled as Ambivalent and Clear Reply tend to contain more words and exhibit greater variability in length compared to Clear Non-Reply. In contrast, Clear Non-Reply responses are generally shorter, with a lower median word count and a more concentrated distribution, although all classes contain several outliers.

Overall, these results suggest that answer length may be related to the clarity label. Shorter answers are more often associated with cases where the politician does not provide a substantial response to the question, while longer answers may include more context, indirect statements, or topic shifts, making it less clear whether the question has been fully answered. This observation suggests that response length may serve as a useful feature for distinguishing between clarity categories.

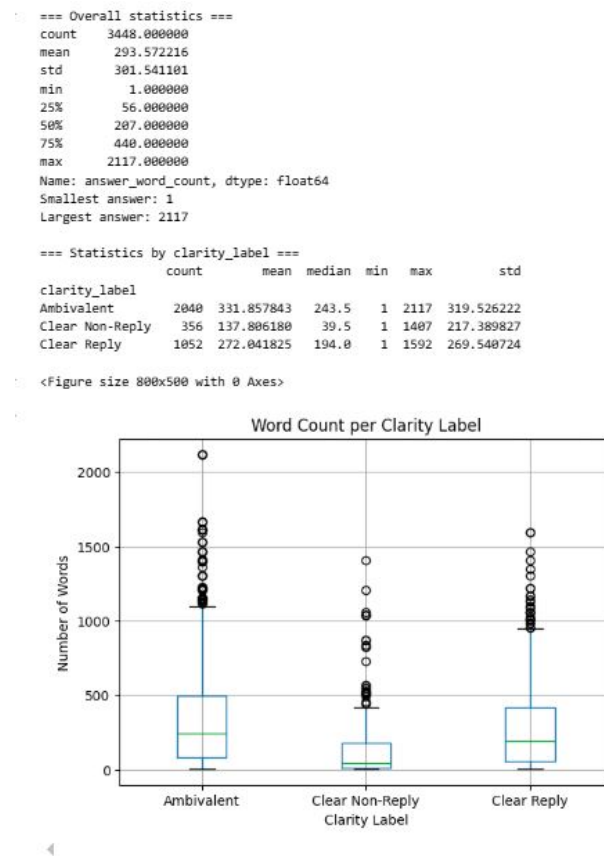


Figure 10: Words per label

Detection of text patterns in the dataset Although it is possible to manually inspect several examples from the dataset (e.g., by printing random entries), such inspection does not guarantee that all instances follow the same format. To systematically verify whether patterns are present in the dataset, a function named `detect_text_patterns` was implemented that scans the textual data and reports the occurrence of common types of noise. Specifically, the function searches for patterns such as HTML tags, email addresses, URLs, Twitter mentions, hashtags, digits, and newline characters using regular expressions. Additionally, it checks for the presence of emojis in the text. This automated inspection provides a quick overview of the dataset and helps determine whether specific cleaning operations are required before further processing. The results of this analysis indicated that the only notable pattern detected in the text column was the presence of numerical values.

```

detect_text_patterns(training_data["text"])
✓ 6.6s

Starting dataset inspection...
-----
Clean: No HTML Tags found.

Clean: No Emails found.

Clean: No URLs (http/www) found.

Clean: No Twitter Mentions (@) found.

Clean: No Hashtags (#) found.

Found: 1527 texts with Digits.
Clean: No Newlines (\n) found.

Clean: No Emojis found.

-----
Inspection completed!

```

Figure 11: Detection Patterns

Detection of spelling errors Moreover, a quality check was conducted to estimate the percentage of spelling errors present in the dataset. To do so, a function was implemented that scans the textual data and compares each word against an English dictionary using a spell-checking library. The function tokenizes the text into individual words, counts the total number of words, and identifies those that are not recognized by the dictionary as potential spelling errors. It then computes the percentage of misspelled words relative to the total number of words in the dataset. In addition, the function outputs a small set of example misspelled words to provide a qualitative indication of the types of spelling inconsistencies present in the data.

As shown in Figure 12, the analysis indicates that spelling errors constitute a very small portion of the dataset (approximately 1.96% of all words), suggesting that the textual data is generally clean and that spelling mistakes are unlikely to significantly affect the performance of the classification models. After removing stopwords, which usually carry limited semantic information for classification tasks, the percentage of misspelled words increased slightly to 2.94%. This suggests that spelling errors are somewhat more common in the more informative parts of the text, although their overall proportion remains relatively low.

```

check_spelling_errors(training_data["text"])
✓ 4.4s

Total words: 1125356
Misspelled words: 22040
Percentage of misspelled words: 1.96%
Example misspelled words: ['thei', 'chineseno', 'aboutit', 've', 'donei']

```

Figure 12: Spelling errors

2.2.4. Token Analysis. This analysis was performed mainly to get an overview of how large the question–answer pairs become once tokenized with the Qwen tokenizer, and to check whether the token-length distribution differs between the training and test

sets. This is a useful check, since a large mismatch in input length between the two sets could affect how reliably the behaviour observed on the training data transfers to the test set.

```
Total samples: 3448
Mean length: 376.04
Median length: 273.0
Maximum length: 2465
Minimum length: 5
```

Figure 13: Tokens

```
Total samples: 308
Mean length: 412.91
Median length: 379.5
Maximum length: 1599
Minimum length: 13
```

Figure 14: Tokens

The token statistics show that the test inputs are slightly longer on average than the training inputs. More specifically, the mean token length is 412.9 tokens for the test set compared to 376.0 tokens for the training set, while the median is also higher in the test set (379.5 vs. 273.0). This suggests a mild shift toward longer answers in the test set.

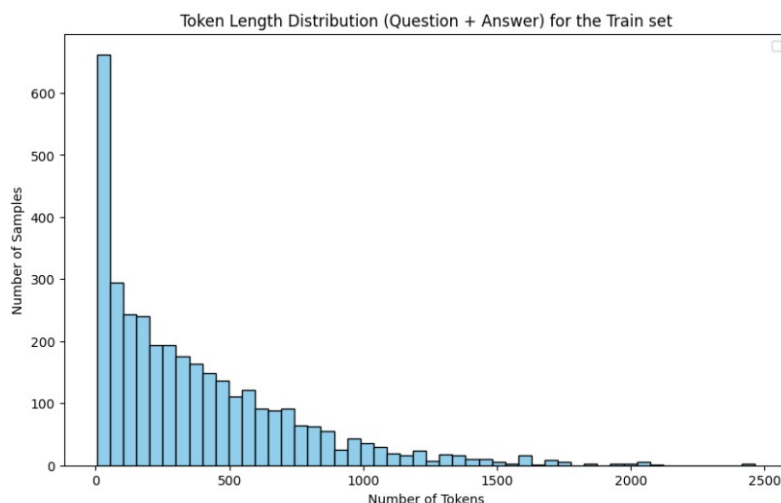


Figure 15: Tokens distribution

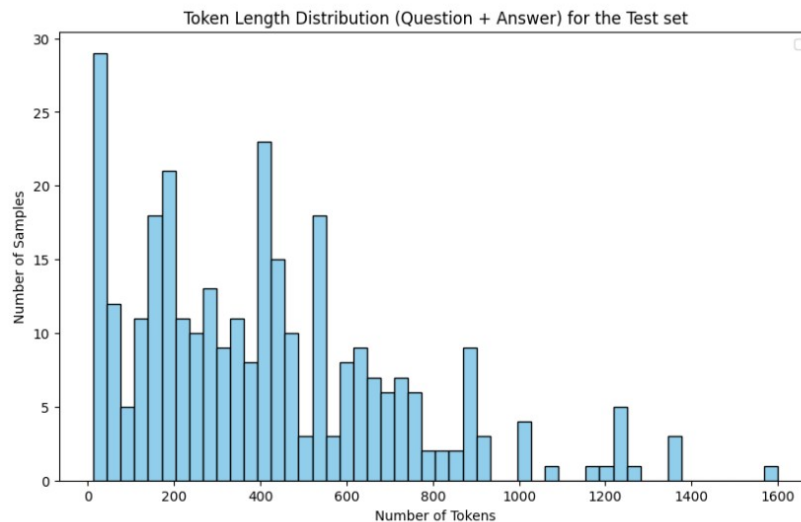


Figure 16: Tokens distribution

The histograms further support this observation, showing that both distributions are right-skewed. Most answers are relatively short, while a smaller number of responses are much longer and form a long tail. This pattern is particularly visible in the training set, where a large number of instances are concentrated in the lower token-length range. In contrast, the test set appears more spread out across medium and longer token lengths, which is consistent with its higher mean and median values.

3. Pre-processing

Based on the findings from the exploratory data analysis, a simple and targeted pre-processing pipeline was adopted. The pattern-detection step showed that the dataset was relatively clean, with no meaningful presence of HTML tags, URLs, hashtags, or Twitter mentions, and the spelling analysis found that misspelled words accounted for less than 2% of the vocabulary. Since instruction-tuned large language models are generally robust to minor spelling variations, punctuation differences, and numerical values, no aggressive cleaning or spelling correction was applied, and the raw question and answer text was kept intact so that the models receive the same input a human reader would.

To reduce redundancy and make the experiments more efficient in terms of run-time, duplicate (question, interview_answer) pairs were removed from the training set. After this step, the training set was reduced from 3,448 to 3,390 rows.

```
Before drop_duplicates: (3448, 21)
After drop_duplicates: (3390, 21)
```

Figure 17: Number of training rows after duplicate removal

Next, the deduplicated dataset was split using an 11% stratified hold-out strategy, in order to evaluate the prompting techniques without information leakage. The split

produced a demonstration pool (pool_df, 3,017 rows), which was used only for retrieving few-shot examples, and an experimental evaluation set (exp_df, 373 rows), which was used for all reported metrics. Keeping these two sets disjoint ensures that no instance shown to a model as a demonstration is also used during evaluation.

Stratification was applied to preserve the original label distribution in the evaluation set, which is particularly important given the class imbalance observed in the dataset. As a result, exp_df contains 221 Ambivalent, 114 Clear Reply, and 38 Clear Non-Reply examples. Ambivalent therefore remains the dominant class, while Clear Non-Reply remains the minority class.

All five models were evaluated on the same experimental set. This makes the comparison fair across different model sizes and families, while also keeping the inference process tractable.

```
Pool:
clarity_label
Ambivalent      1783
Clear Reply      923
Clear Non-Reply  311
Name: count, dtype: int64

Experiment dataset:
clarity_label
Ambivalent      221
Clear Reply     114
Clear Non-Reply  38
Name: count, dtype: int64
```

	count	min	max	mean	median
clarity_label					
Ambivalent	221	1	1667	342.1	243.0
Clear Non-Reply	38	2	514	100.8	51.5
Clear Reply	114	1	1170	296.2	223.0

Figure 18: Pool and Experiment datasets

The following figure shows the answer length distribution for each class in the experimental dataset. The statistics indicate noticeable differences between the three clarity labels. On average, Ambivalent answers are the longest, Clear Reply answers have an intermediate length, and Clear Non-Reply answers are the shortest.

This pattern is consistent with the findings of the exploratory data analysis, where the Ambivalent class was also associated with longer answers, while Clear Non-Reply examples tended to contain fewer words.

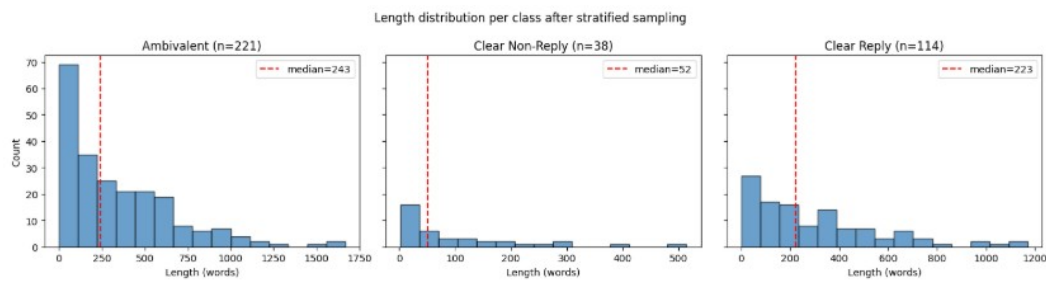


Figure 19: The answer length distribution per each class

4. Models

LLMs Large Language Models (LLMs) are artificial-intelligence systems trained on very large amounts of textual data with the objective of predicting the next token in a sequence. Most LLMs are based on neural networks, and a key element behind their strong performance is the attention mechanism, which allows the model to focus on different parts of the input when producing each part of the output.

Prompting Prompting is the technique of providing instructions in natural language to an LLM in order to guide its output. Its main advantage is that the task is specified entirely through the prompt, so the model can be steered toward a particular problem without any additional training or fine-tuning. As a consequence, the wording, structure, and content of the prompt can strongly affect the quality and consistency of the response. In this work, prompting is used for a classification task: for each question-answer pair, the model must predict one of three clarity labels — Clear Reply, Ambivalent, or Clear Non-Reply. Since no fine-tuning is performed, the task definition, the label descriptions, and the expected output format are all provided through the prompt.

Models The evaluation was conducted on five instruction-tuned LLMs: three models from the Qwen3.5 family and two models from the Gemma 3 family. The Qwen3.5 models are mainly used to study the effect of model size within the same model family, while the Gemma 3 models are included for comparison with a different model family. The technical specifications of all models are summarized in Table 2.

All models are used in their instruction-tuned form, without any additional fine-tuning. Each prompt follows the model’s native chat format and is structured using a system message and a user message.

Table 2: Detailed technical specifications of the evaluated language models.

Model Name	Total Parameters	Context Length	Data Type	Vocab Size
Qwen/Qwen3.5-0.8B	752,393,024 (0.75B)	262,144 tokens	bfloat16	248,320
Qwen/Qwen3.5-2B	1,881,825,088 (1.88B)	262,144 tokens	bfloat16	248,320
Qwen/Qwen3.5-4B	4,205,751,296 (4.21B)	262,144 tokens	bfloat16	248,320
google/gemma-3-1b-it	999,885,952 (1.00B)	131,072 tokens*	bfloat16*	256,000*
google/gemma-3-4b-it	4,300,079,472 (4.30B)	131,072 tokens*	bfloat16*	256,000*

4.1. Prompting Techniques

In this section, the prompting techniques explored in the experiments are presented along with their prompt formulations.

4.1.1. Zero-shot Prompting. Zero-shot prompting is a prompting strategy where the model is asked to perform a task without being given any labelled examples. The model relies only on the task instruction, the label descriptions, and its pretrained knowledge to produce the output.

The baseline system prompt is the following:

```
You are an expert classifier.
Given the interview question and the president's answer, classify the answer
into exactly one of the following labels:
Labels:
- Clear Reply
- Ambivalent
- Clear Non-Reply

Output the label only.
```

The test instance is always passed in the user turn using the following fixed template:

```
Question: ...
Answer: ...
```

Three prompt-design variants were compared against the baseline in order to examine how the wording and structure of the prompt affect the classification results. The simplified variant removes the “expert classifier” framing and keeps only the task description together with the list of valid labels.

```
Given the interview question and the president's answer, classify the answer
into exactly one of the following labels:
Labels:
- Clear Reply
- Ambivalent
- Clear Non-Reply

Output the label only.
```

The detailed variant adds explicit definitions for each label, as well as a small set of context rules based on the findings of the exploratory data analysis.

You are an expert classifier.
 Given the interview question and a president's answer, classify the answer into exactly one of the following labels:
 Labels:
 - Clear Reply: The president directly and clearly answers the specific question.
 - Ambivalent: It is unclear or ambiguous whether the president actually answered the specific question.
 - Clear Non-Reply: The president completely avoids, ignores, or fails to answer the question.

Context Rules:
 1. The provided question may be a sub-question extracted from a larger interview.
 2. If the president's answer covers multiple topics, evaluate only the specific portion of the answer that is relevant to the given question.
 3. Usually, longer answers are Ambivalent and shorter answers are Clear Non-Reply.
 Output the label only.

Finally, the no-system-prompt variant uses the same instruction text, but instead of placing it in a separate system message it concatenates it in front of the question and answer inside a single user turn. This was used to check whether the model relies on the system role or only on the instruction content.

(user) You are an expert classifier.
 Given the interview question and the president's answer,
 classify the answer into exactly one of the following labels:
 ...
 Output the label only.
 Question: <test question>
 Answer: <test answer>

4.1.2. Few-shot Prompting. Few-shot prompting is a prompting strategy where the model is given a small number of labelled examples before making a prediction for a new instance. These examples act as demonstrations and help the model understand both the expected output format and the relationship between the question-answer pair and the clarity label.

The examples used in the prompt can play an important role in the performance of the model. For this reason, two selection methods were designed and compared: random selection and semantic kNN selection.

Random selection In the random selection setting, the examples are chosen randomly from `pool_df`. The only constraint is that the prompt contains one example from each class, so that every prompt includes one Clear Reply, one Ambivalent, and one Clear Non-Reply demonstration. The system prompt is the same as in the zero-shot baseline, and each demonstration is added as a user turn followed by an assistant

turn containing only the gold label. After the demonstrations, the test question–answer pair is given to the model, which must predict one of the three valid clarity labels.

```
(system)    You are an expert classifier. ... Output the label only.
(user)      Question: <example question>
            Answer:  <example answer>
(assistant) <example label>
... (one demonstration per class) ...
(user)      Question: <test question>
            Answer:  <test answer>
```

Semantic kNN-based Few-shot Selection The second selection method keeps the same few-shot prompt structure described above, but changes the way in which the demonstrations are selected. Instead of choosing examples randomly, the method selects examples that are semantically similar to the test instance. This follows a k-nearest neighbours (kNN) idea, where the nearest examples are those with the highest semantic similarity to the current input.

Each question–answer pair is first converted into a single text representation using the format Question: ... and Answer: These texts are then encoded into normalised dense vectors using the sentence-embedding model all-MiniLM-L6-v2. The embeddings of all examples in the demonstration pool are computed once and stored. For each test instance, its embedding is compared with all pool embeddings using cosine similarity, implemented as the dot product between normalised vectors. The pool examples are then ranked from the most similar to the least similar.

Two variants of this method are tested. In the balanced variant, the ranked list is scanned until the most similar example from each of the three classes is selected, and any remaining slots are filled with the next most similar examples. This prevents the prompt from containing examples that all belong to the same class. In the unbalanced variant, the method simply selects the top- K most similar examples, regardless of their class label.

The semantic kNN approach is tested with $K = 3$ and $K = 6$. Larger values of K were not used due to memory constraints.

4.1.3. Chain-of-Thought Prompting. Chain-of-Thought prompting is a prompting strategy where the model is explicitly instructed to produce an intermediate reasoning step before giving the final answer. In classification tasks, this can help the model analyse the input more carefully before selecting a label.

In this work, Chain-of-Thought prompting is activated through the prompt by requiring the model to first explain how the president’s answer relates to the specific question and then output the final clarity label. The output is constrained to a fixed two-line format, with a short reasoning step followed by one of the three valid labels. The system prompt is shown below:

```
You are an expert classifier.
Given a question and a president’s answer, classify the answer into exactly
one of the following labels:
Labels:
- Clear Reply
- Ambivalent
```

```
- Clear Non-Reply

Output format:
Reasoning: <2-4 sentences explaining how the answer relates to the specific
          sub-question>
Label: <Clear Reply | Ambivalent | Clear Non-Reply>
```

The explicit Reasoning: and Label: fields make the output easier to parse after generation. Although the model is allowed to produce a short reasoning step, only the final label is used for evaluation.

4.1.4. Chain-of-Thought with Few-shot Prompting. This technique combines Chain-of-Thought prompting with few-shot demonstrations. The system prompt is the same as in the CoT setting, but each demonstration also contains a short reasoning step before the final label. In this way, the assistant turns follow the same Reasoning: ... / Label: ... format that the model is expected to produce.

The reasoning steps used in the demonstrations were generated automatically by the same model using the CoT prompt. For each selected few-shot example, the model was first asked to produce a short reasoning explanation, and this generated reasoning was then combined with the gold label to form the final demonstration message.

```
(user)      Question: <example question>
              Answer:  <example answer>
(assistant) Reasoning: <short reasoning for this example>
              Label:   <example label>
```

In this way, the model is shown not only the correct label, but also the type of reasoning that should lead to it.

4.1.5. Self-Consistency. Self-consistency is a prompting strategy where the model generates multiple responses for the same input instead of producing only one prediction. The final answer is then selected by aggregating these responses, usually through majority voting. The idea is that if the model is uncertain, different generations may lead to different labels, while a stable prediction is more likely to appear consistently across multiple outputs.

In this work, self-consistency is applied on top of the Chain-of-Thought prompt: the model is asked to classify the same question–answer pair multiple times using the CoT instruction, and the final prediction is obtained by majority voting over the produced labels. To encourage diversity across the repeated generations, sampled decoding is used instead of deterministic decoding. Each generated response is parsed to extract the predicted label, and the final prediction is selected as the label that appears most frequently. This allows us to examine whether repeated generations can improve the stability of the classification results compared to a single model output. The specific sampling settings are described in the experimental implementation section.

5. Evaluation Metrics

To evaluate the performance of the LLMs on the clarity classification task, the parsed model predictions were compared against the gold clarity labels. The evaluation was

based on standard classification metrics. A detailed classification report was produced for the three valid classes: Clear Reply, Ambivalent, and Clear Non-Reply. This report includes precision, recall, and F1-score for each class, as well as macro and weighted averages. In addition, we report the main summary metrics separately: accuracy, macro-F1, weighted-F1, macro precision, and macro recall.

Since the LLMs are used as generative models, their outputs are not automatically restricted to one of the predefined classes. Instead, a model may produce free-form text, which may include the requested label, an explanation, or an output that does not follow the expected format. For this reason, each raw model response is first parsed and mapped to one of the three valid labels defined above. If none of these labels can be detected in the response, the prediction is recorded as Invalid.

Handling of Invalid predictions. The way Invalid predictions affect the evaluation depends on the metric. Accuracy is computed over all samples, so every Invalid prediction is counted as incorrect. This makes the evaluation stricter, since models are rewarded not only for choosing the correct class, but also for following the required output format.

For the F1-based metrics, Invalid is not introduced as a fourth class: macro-F1, weighted-F1, macro precision, and macro recall are computed only over the three valid labels. However, an Invalid prediction still penalises the model, because it is counted as a false negative for the corresponding true class and therefore lowers the recall and F1-score of that class.

Among the F1-based metrics, macro-F1 is treated as the most informative for this task because the dataset is imbalanced. The Ambivalent class is the majority class, while Clear Non-Reply is the minority class. By giving equal weight to all classes, macro-F1 reflects performance across the full label set rather than rewarding strong performance on the dominant class. Weighted-F1 is also reported, since it is the official metric of the associated Kaggle competition and the score on which the final system is ranked. However, because it weights each class by its frequency, a model can still obtain a relatively high weighted-F1 even if it performs poorly on the minority class.

Confusion matrix. The confusion matrix is used to inspect the types of mistakes made by each model. When at least one Invalid prediction is produced, an additional Invalid category is included on the predicted axis. The rows remain the three true classes, since the gold labels never contain Invalid. This makes it possible to observe which true classes most often lead to invalid generations and whether a model tends to fail more on specific labels.

Error analysis. A finer analysis of the mistakes is also produced. Errors are first decomposed into wrong-label errors, where the model predicts one of the three valid labels but the prediction is incorrect, and invalid-format errors, where the model fails to produce a valid label. This distinction helps reveal whether a model mainly fails because it confuses the classes or because it does not follow the required output format. In addition, the most frequent confusion patterns (true $\rightarrow$ predicted) are listed, and a small random sample of the raw invalid outputs is inspected qualitatively in order to characterise typical failure modes.

Accuracy by answer length. Finally, accuracy is analysed with respect to answer length. The interview answers are grouped into four quantile-based buckets of roughly equal size, where length is measured in words. Per-bucket accuracy is then compared against the overall accuracy, which is used as a reference. This analysis is used to examine whether the models perform differently on shorter and longer answers, since longer political answers may contain more context, topic shifts, or ambiguous statements that make classification harder.

6. Experiments

The experimental evaluation includes five instruction-tuned LLMs: Qwen3.5-0.8B, Qwen3.5-2B, Qwen3.5-4B, Gemma 3 1B, and Gemma 3 4B. The experiments are organised into three main parts.

1. **Prompting strategy comparison on Qwen3.5-0.8B.** The first part focuses on the smallest model, Qwen3.5-0.8B, in order to examine the effect of the prompting strategy while keeping the model fixed. Different zero-shot prompt formulations are tested, including a simplified prompt, a more detailed prompt, and a version where the instruction is provided only in the user message. In addition, few-shot prompting, semantic kNN few-shot prompting, Chain-of-Thought, CoT with few-shot demonstrations, and Self-Consistency are evaluated. This part is used to identify which prompting strategies are most effective before comparing larger models.
2. **Model size comparison within the Qwen3.5 family.** The second part compares three Qwen3.5 models of different sizes: Qwen3.5-0.8B, Qwen3.5-2B, and Qwen3.5-4B. Since these models belong to the same family, this setup allows us to examine how model size affects performance on the clarity classification task. The main comparison focuses on the baseline zero-shot prompt and the best semantic kNN few-shot setting. Chain-of-Thought was also tested as an exploratory setting, but it was not used as the main comparison method because of its higher computational cost and weaker performance.
3. **Comparison with another model family.** The third part includes two models from the Gemma 3 family: Gemma 3 1B and Gemma 3 4B. These models are used to examine whether the behaviour observed in the Qwen3.5 experiments also appears in a different model family. This comparison focuses on selected prompting techniques, mainly zero-shot and semantic kNN few-shot prompting, so that the effect of model family and scale can be studied without making the experimental setup unnecessarily large.

6.1. Implementation Details

All models were used through the Hugging Face transformers library, loaded as pre-trained, instruction-tuned checkpoints without any fine-tuning. The Qwen3.5 models were loaded with AutoModelForCausalLM using automatic precision selection (torch_dtype="auto") and automatic device placement (device_map="auto"), so that the weights are placed on the available hardware efficiently. The Gemma 3 4B model (google/gemma-3-4b-it) was loaded with Gemma3ForConditionalGeneration in bfloat16

precision, again with `device_map="auto"`, and its tokenizer was obtained from the corresponding processor. The smaller Gemma 3 1B model (`google/gemma-3-1b-it`) was instead loaded with `AutoModelForCausalLM` and its own tokenizer in `bfloat16` precision.

Inference was performed in small batches (a batch size of 2–4 was used, depending on the model and the available memory). The semantic-kNN few-shot experiments are an exception: there each test instance was processed on its own, because every instance retrieves its own set of demonstrations, so its prompt differs from the others and cannot share a batch. Because the question–answer pairs differ in length, the tokenizer pads them to a common length within each batch. Padding is applied on the left: since the models are decoder-only and generate text to the right of the prompt, left padding keeps the real tokens right-aligned, so that across the batch the newly generated tokens all start at the same position. When a tokenizer did not define a padding token, the end-of-sequence token was reused as the padding token.

Every prompt was built with the model’s own chat template (`apply_chat_template` with `add_generation_prompt=True`), which appends the assistant-turn header so that the model continues as the assistant. The thinking mode was disabled (`enable_thinking=False`) so that the model produces the final answer directly rather than an internal reasoning trace. For most models the internal `<think>` tag was closed correctly even without an explicit setting. For Qwen3.5-4B, however, it was not always closed, so part of the generation budget was consumed by reasoning text and the model did not reach the final label within the allowed length. To address this, `enable_thinking` was set to `False` explicitly in the classification functions, and the maximum number of generated tokens was increased; this made the output more stable and ensured that both the reasoning (when required) and the final label were produced.

The exact structure of the resulting prompt depends on each model’s chat template. Figure 20 shows the prompt produced for Qwen3.5, where the system instruction is preserved as a separate system turn, while Figure 21 shows the corresponding prompt for Gemma 3, where the system content is automatically merged into the first user turn because Gemma does not support a native system role. The instruction content is identical in both cases; only its placement within the chat format differs.

```

<|im_start|>system
You are an expert classifier.
Given the interview question and the president's answer, classify the answer into exactly one
of the following labels:
Labels:
- Clear Reply
- Ambivalent
- Clear Non-Reply

Output the label only.<|im_end|>
<|im_start|>user
Question: Will the United States recognize it?
Answer: I suggest you study the Ahtisaari plan. Not to be, like, the, you know, grumpy guy. We
ll, how to get- how do we get more peacekeepers into Darfur? Of course, what has been delaying
getting more peacekeepers has been the lack of understanding between the U.N. and the Governme
nt of Sudan on the status of forces agreement. Now that one has been signed, I think it has cl
eared the way. There are many volunteers, and as Chair of the African Union, we certainly seek
out more and more volunteers as they are needed. We use our good offices to see more and more
African countries contribute. We have volunteered to contribute one battalion. If there is nee
d for another one, we will certainly do that.<|im_end|>
<|im_start|>assistant
<think>

</think>

- Clear Non-Reply<|im_end|>
<|endoftext|>

```

Figure 20: Chat template applied to Qwen3.5: the system message is preserved as a separate system turn.

```

<bos><bos><start_of_turn>user
You are an expert classifier.
Given the interview question and the president's answer, classify the answer into exactly one
of the following labels:
Labels:
- Clear Reply
- Ambivalent
- Clear Non-Reply

Output the label only.

Question: Will the United States recognize it?
Answer: I suggest you study the Ahtisaari plan. Not to be, like, the, you know, grumpy guy. We
ll, how to get- how do we get more peacekeepers into Darfur? Of course, what has been delaying
getting more peacekeepers has been the lack of understanding between the U.N. and the Governme
nt of Sudan on the status of forces agreement. Now that one has been signed, I think it has cl
eared the way. There are many volunteers, and as Chair of the African Union, we certainly seek
out more and more volunteers as they are needed. We use our good offices to see more and more
African countries contribute. We have volunteered to contribute one battalion. If there is nee
d for another one, we will certainly do that.<end_of_turn>
<start_of_turn>model
Ambivalent<end_of_turn>

```

Figure 21: Chat template applied to Gemma 3: the system content is automatically merged into the first user turn, since Gemma does not support a native system role.

Reproducibility For reproducibility, a global random seed of 42 was fixed across Python’s random module, NumPy, and PyTorch (including CUDA), and the same seed was used for the stratified split and for sampling the few-shot demonstrations. All five models were evaluated on the same held-out evaluation set of 373 instances, so that the observed differences reflect model size and prompting strategy rather than differences in the evaluation data.

Decoding settings. The decoding strategy was adapted to each prompting method. For zero-shot and few-shot prompting, where only a single label is expected, greedy decoding was used, which makes the output deterministic and reproducible. The maximum number of new tokens was kept very small (20 tokens), since any of the three valid labels fits well within this budget; this also discourages the model from generating additional text beyond the label itself. For Chain-of-Thought and Chain-of-Thought with few-shot, greedy decoding was again used, but with a larger budget of 300 new tokens, so that the model has enough room to produce both the short reasoning step and the final label. Self-Consistency is the only method that uses sampling: the model is queried three times with temperature = 1 and top- p = 0.9, and the final label is obtained by majority vote over the valid predictions.

Prompting method	Decoding	Sampling parameters
Zero-shot	Greedy	≤ 20 new tokens (label only)
Few-shot (random / kNN)	Greedy	≤ 20 new tokens (label only)
Chain-of-Thought	Greedy	300 new tokens (reasoning + label)
CoT + Few-shot	Greedy	300 new tokens (reasoning + label)
Self-Consistency	Sampling	$T = 1$, top- $p = 0.9$, 3 samples, majority vote

Table 3: Decoding settings used for each prompting method.

6.2. Output Parsing and Label Mapping

Because the models generate free text, every raw output is converted into one of the three labels (or Invalid) by a parsing step. Two parsers are used, depending on the prompting method.

For zero-shot and few-shot outputs, where the model should return only the label, the parser works as follows. If the output contains a thinking trace, that is a `</think>` tag, everything up to and including the last `</think>` is discarded and only the text after it is kept; the remaining text is then stripped of surrounding whitespace. The parser then searches this text for one of the three valid labels, checking them from the longest to the shortest string (Clear Non-Reply, then Clear Reply, then Ambivalent); this ordering is a safeguard so that a shorter label is not matched as part of a longer one (for example, matching Clear Reply inside Clear Non-Reply). The comparison is case-insensitive, and the first label found in the text is returned. If none of the three labels appears anywhere in the text, the output is recorded as Invalid.

For Chain-of-Thought outputs, a stricter parser is used, because the model is expected to follow the Reasoning: ... / Label: ... format. As before, any text up to the last `</think>` is removed first. The parser then uses a regular expression to find the pattern Label: followed by one of the three valid labels (case-insensitive). When a

match is found, the value is normalised to the canonical spelling: if it contains “non” it becomes Clear Non-Reply, if it contains “ambiv” it becomes Ambivalent, otherwise it is treated as Clear Reply. If no Label: line is found, the prediction is set to Invalid, since this means the model did not follow the required format. The reasoning part is also extracted separately, as the text between the Reasoning: marker and the Label: marker (or the end of the output), and is stored for inspection.

Although the parsing rules were initially designed based on the output behaviour of the Qwen3.5 models, the same parsers were applied without any modification to the Gemma 3 outputs and extracted the label correctly in both cases. This suggests that the parsing logic is robust to differences in chat templates and tokenization between the two families, and that no model-specific post-processing is required.

6.3. Experimental Results

6.3.1. Qwen3.5-0.8B Experiments.

Zero-shot (baseline). The baseline zero-shot prompt, described above, gives the model only the task instruction and the question–answer pair, with no examples. On Qwen3.5-0.8B it reached a macro-F1 of 0.447, a weighted-F1 of 0.520 and accuracy of 0.515

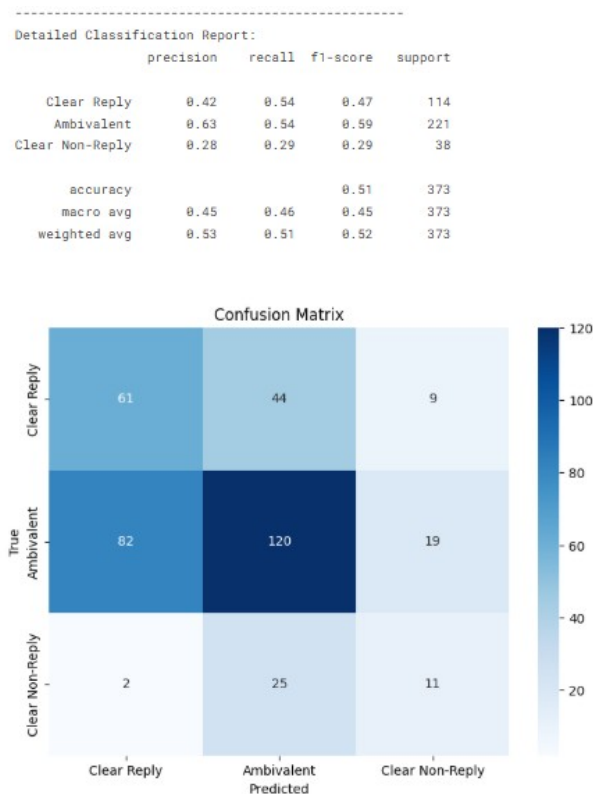


Figure 22: Confusion Matrix Zero Shot Baseline

This experiment achieves one of the strongest overall performances among all experiment. More importantly, it obtains a high macro-F1 score, which suggests that

the improvement is not mainly due to better classification of the majority class. Since macro-F1 gives equal weight to all classes, this result indicates that the model separates the three clarity labels more effectively.

For this reason, this experiment is used as an important reference point for the following comparisons.

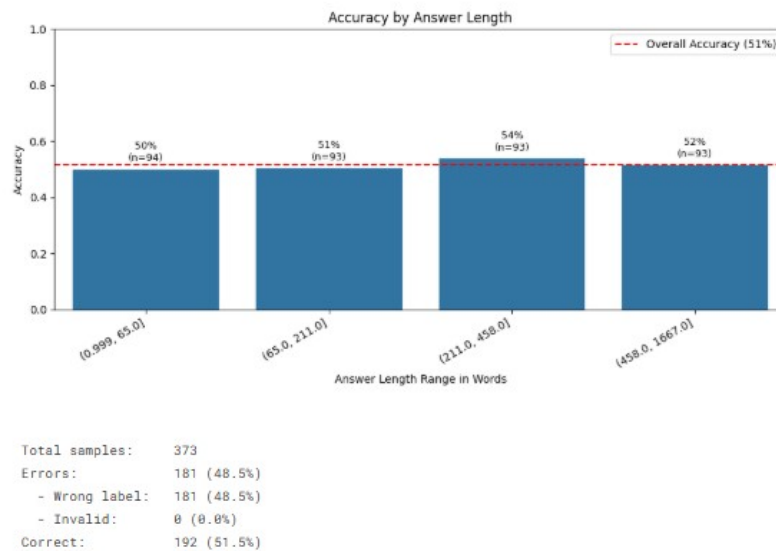


Figure 23: Accuracy per Length size

As shown in the figure, the accuracy-by-answer-length analysis does not reveal a clear systematic difference across the answer-length groups. In addition, no invalid predictions were produced in this configuration.

Zero-shot (simplified). This variant keeps only the task description and the label list, removing the “expert classifier” framing, in order to measure how much that framing contributes on its own. The scores dropped to accuracy 0.357, macro-F1 0.314, and weighted-F1 0.321. The clear drop shows that even a short role description helps a small model, which is sensitive to the exact wording of the prompt.

Detailed Classification Report:

	precision	recall	f1-score	support
Clear Reply	0.35	0.75	0.47	114
Ambivalent	0.55	0.17	0.26	221
Clear Non-Reply	0.17	0.26	0.20	38
accuracy			0.36	373
macro avg	0.36	0.39	0.31	373
weighted avg	0.45	0.36	0.32	373

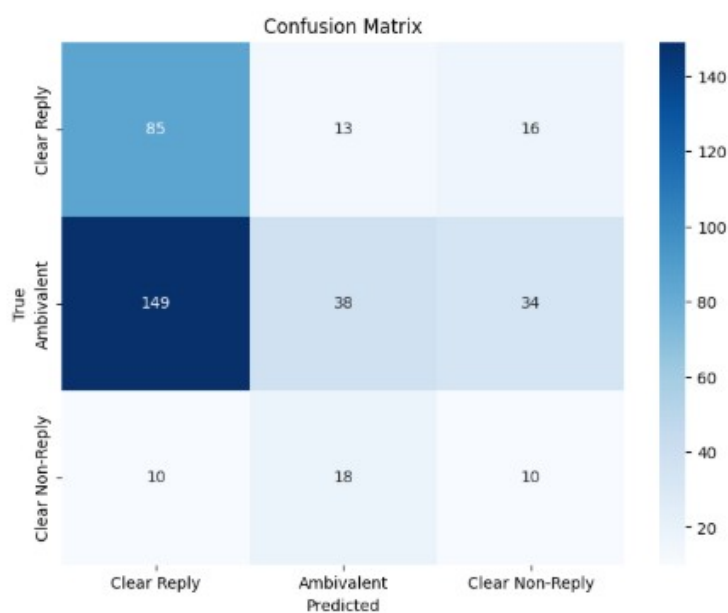


Figure 24: Confusion Matrix Zero Shot Simplified prompt

The model classified most samples as Clear Reply, indicating a strong prediction bias toward this class.

Zero-shot (detailed). Here an explicit definition of each label and a few heuristic rules (e.g. “longer answers are usually Ambivalent”) were added, to test whether richer instructions help. Instead, performance fell to the lowest zero-shot level: accuracy 0.306, macro-F1 0.266, weighted-F1 0.221.

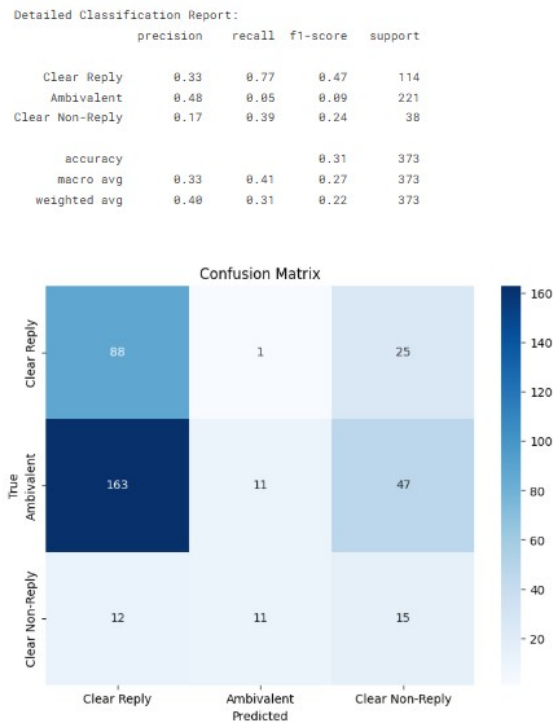


Figure 25: Confusion Matrix Zero Shot Detailed prompt

The injected “length” rule backfired and pushed the model to over-predict Clear Reply, so the extra instructions acted as a bias rather than as guidance.

Zero-shot (only user message). In this version, the same instruction is placed entirely inside the user message instead of being provided as a separate system message. This experiment was designed to examine whether the model benefits from receiving the task instruction through the system role. The model reached an accuracy of 0.389, a macro-F1 of 0.300, and a weighted-F1 of 0.372.

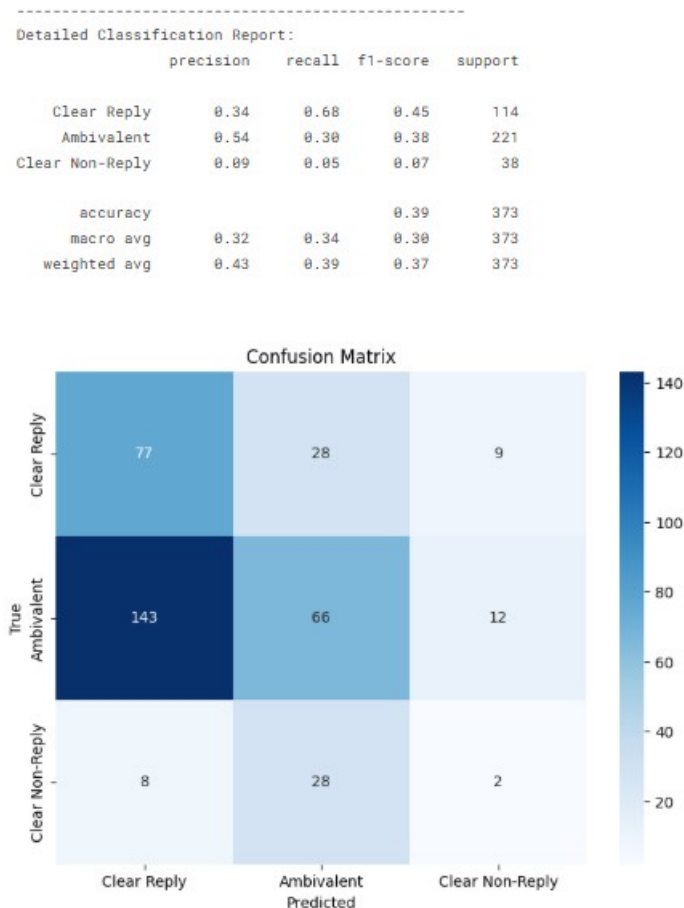


Figure 26: Confusion Matrix Zero Shot User message only

The drop in performance compared with the zero-shot baseline shows that, for Qwen3.5-0.8B, the placement of the instruction matters. Since this model supports a separate system message, providing the classifier instruction in that role appears to help the model follow the task more reliably. When the same instruction is moved into the user message, the model becomes less stable and classifies most samples as Clear Reply, indicating a stronger label bias. This suggests that the system message is useful for this model and that the chat-template structure can affect classification performance.

Few-shot (random selection). Few-shot prompting adds demonstrations before the test instance. In this version, one example per class is randomly selected from `pool_df`, so that the prompt contains one Clear Reply, one Ambivalent, and one Clear Non-Reply demonstration.

This setting achieves the highest accuracy so far (0.563), but it also gives the lowest macro-F1 score (0.274), with a weighted-F1 of 0.459. This indicates that the improvement in accuracy does not reflect better performance across all classes. Instead, the model tends to predict the majority class, Ambivalent, for most instances. As a result, it performs relatively well on the dominant class but fails to distinguish the minority classes effectively, which explains the low macro-F1 score.

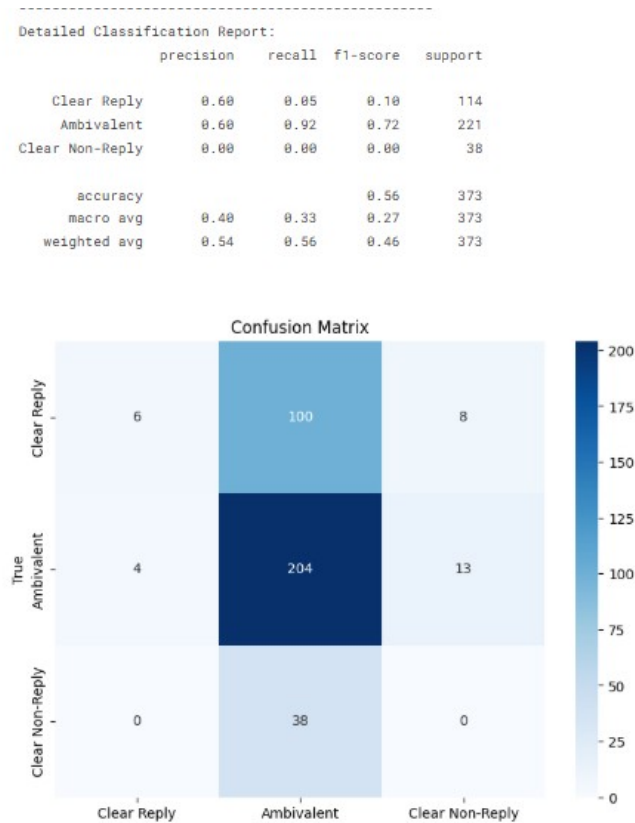


Figure 27: Confusion Matrix Few Shot Random samples

The confusion matrix shows that the model predicts almost all instances as Ambivalent.

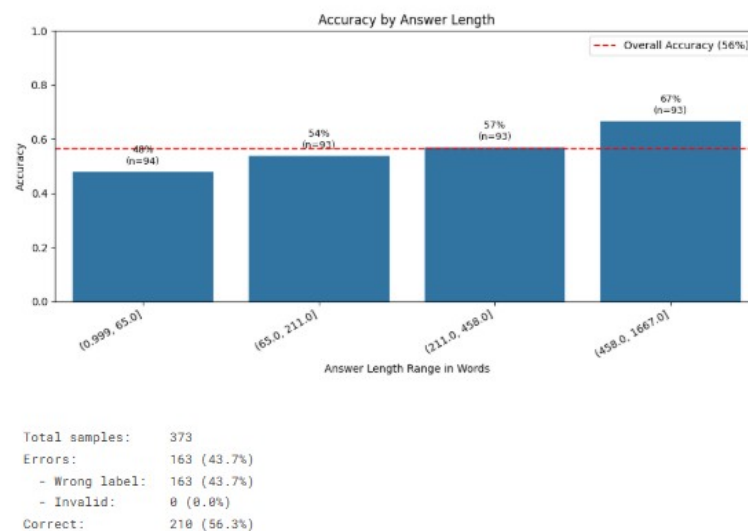


Figure 28: Accuracy per Length size

The accuracy-by-answer-length analysis further supports this interpretation. Based on the exploratory data analysis, Ambivalent answers tend to be longer on average than the other classes. Since this model mostly predicts the majority class, it achieves higher accuracy on longer answers, where Ambivalent instances are more frequent. Therefore, the improved performance on longer responses should not be interpreted

as evidence that the model handles long answers better, but rather as another effect of the class imbalance and its tendency to overpredict Ambivalent.

Few-shot (semantic kNN, K= 3, balanced). In this experiment, the few-shot examples are selected based on semantic similarity to the test instance, while also being constrained to cover all three classes. In contrast to the random few-shot setting, the demonstrations are therefore both relevant to the current input and class-balanced.

This setting partially corrects the majority-class collapse observed in the previous experiment. Macro-F1 increases to 0.382, (still lower from baseline) while accuracy reaches 0.536 and weighted-F1 reaches 0.513. This suggests that using semantically relevant and class-balanced demonstrations helps the model distinguish the classes better than random demonstrations.

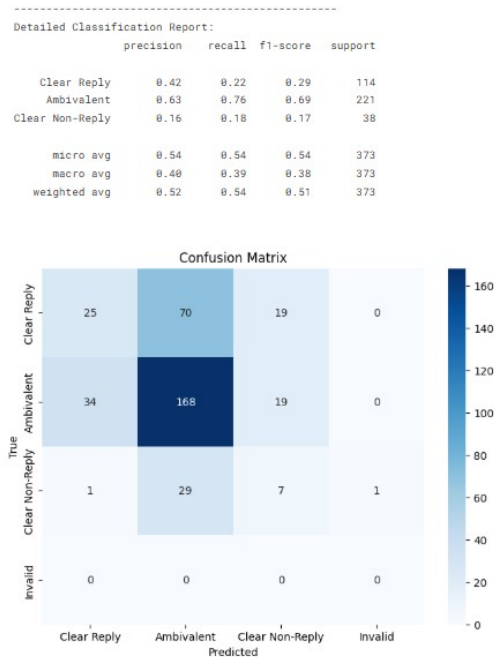


Figure 29: Confusion Matrix Few-shot (semantic kNN, K= 3, balanced)

However, the confusion matrix shows that many Clear Reply examples are still classified as Ambivalent. The same pattern is also observed for several Clear Non-Reply examples. This indicates that, although the semantic kNN selection improves performance compared to random few-shot selection, the model still tends to overpredict the majority class. Therefore, the relatively high accuracy and weighted-F1 are still partly influenced by the dominance of the Ambivalent class, rather than by fully reliable separation between all three labels.

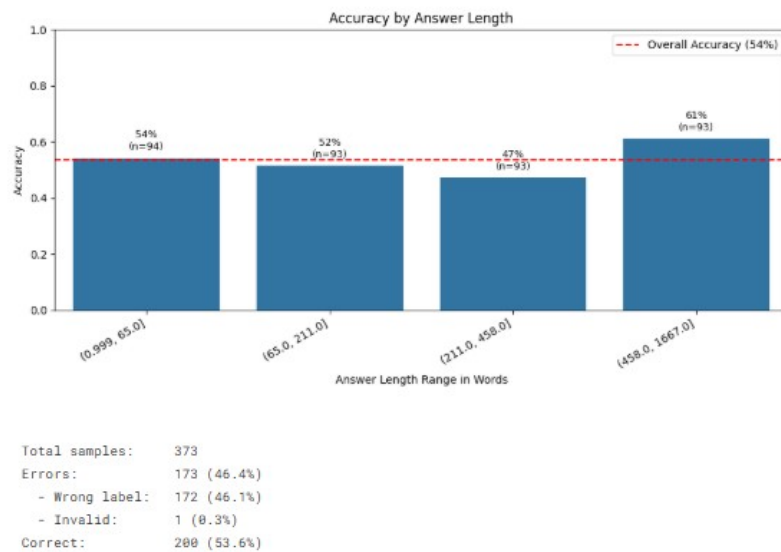


Figure 30: Accuracy per Length size

The model produced one Invalid prediction.

```
Sample of Invalid raw outputs (1 of 1):
[1] (true: Clear Non-Reply) 'No'
```

Figure 31: Invalid output

Few-shot (semantic kNN, K= 3, unbalanced). This version selects the three most semantically similar examples from `pool_df`, regardless of their class label. It was the best configuration on the Qwen3.5-0.8B model in terms of the official evaluation criterion, achieving the highest weighted-F1 (0.538) and the highest accuracy (0.582). Its macro-F1 was lower (0.416), meaning that the improvement was not equally distributed across all classes. However, since the Kaggle leaderboard is based on weighted-F1, this configuration is particularly important for the final submission.

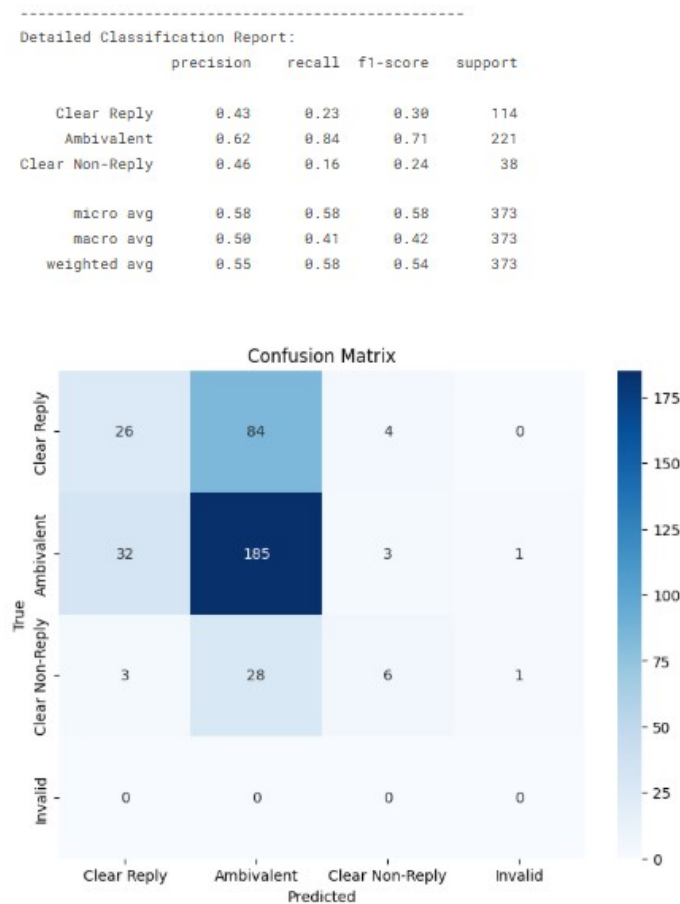


Figure 32: Confusion matrix for semantic kNN few-shot prompting with K= 3 unbalanced examples.

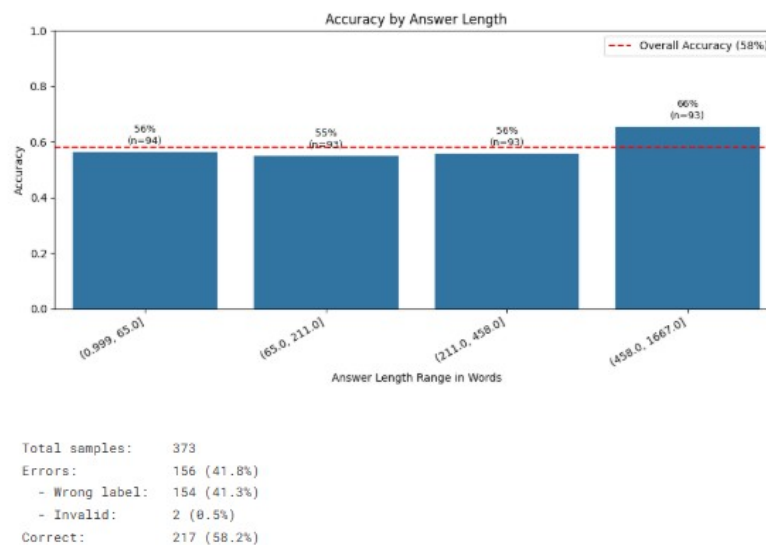


Figure 33: Accuracy by answer length

The model produced two Invalid predictions, so the invalid-output rate remained

very low.

```
Sample of Invalid raw outputs (2 of 2):
[1] (true: Ambivalent) 'Yes'
[2] (true: Clear Non-Reply) 'No'
```

Figure 34: Invalid outputs produced by the semantic kNN few-shot $K=3$ unbalanced setting.

From the confusion matrix and the classification report, it can be observed that the model performs best on the *Ambivalent* class. The number of errors for this class is reduced, and its F1-score reaches 0.71, which is clearly higher than the scores of the other two classes. Since *Ambivalent* is the majority class, this strong performance has a large effect on the weighted-F1 score.

However, the performance on the remaining classes is weaker. The F1-score for *Clear Reply* is around 0.30, while for *Clear Non-Reply* it is only around 0.24. This explains why the macro-F1 remains lower than the zero-shot baseline: the model is not equally strong across all three classes. Nevertheless, because weighted-F1 gives more weight to the more frequent classes, the strong performance on *Ambivalent* makes this configuration the best choice under the Kaggle metric.

Therefore, this experiment represents a trade-off. It is not the most balanced 0.8B configuration in terms of macro-F1, but it achieves the best weighted-F1 and accuracy, while remaining computationally cheaper than the larger models. For this reason, it was selected as the final submission configuration.

Few-shot (semantic kNN, $K=6$, balanced). The balanced $K=6$ semantic kNN few-shot setting achieved a weighted-F1 of 0.53 and a macro-F1 of 0.43. Compared with the smaller $K=3$ balanced setting, increasing the number of demonstrations improved the macro-F1 score while preserving class balance, since the prompt included at least one example from each class and then added additional examples that were semantically similar to the test instance. This result supports the conclusion that the choice and number of demonstrations play an important role in few-shot prompting, especially for balanced performance across classes.

However, although $K=6$ improved macro-F1 compared with the other few-shot variants, it still did not outperform the zero-shot baseline, which achieved a macro-F1 of 0.447. In addition, its weighted-F1 was similar to the smaller $K=3$ setting, while requiring considerably more runtime. Therefore, this configuration was not kept included in the main final run.

Chain-of-Thought. The Chain-of-Thought setting does not improve the performance of the smallest model. Although the prompt encourages the model to reason before producing the final label, the results remain weak, with macro-F1 0.290 and weighted-F1 0.223.

Detailed Classification Report:

	precision	recall	f1-score	support
Clear Reply	0.35	0.77	0.48	114
Ambivalent	0.50	0.04	0.08	221
Clear Non-Reply	0.22	0.58	0.31	38
accuracy			0.32	373
macro avg	0.35	0.46	0.29	373
weighted avg	0.42	0.32	0.22	373

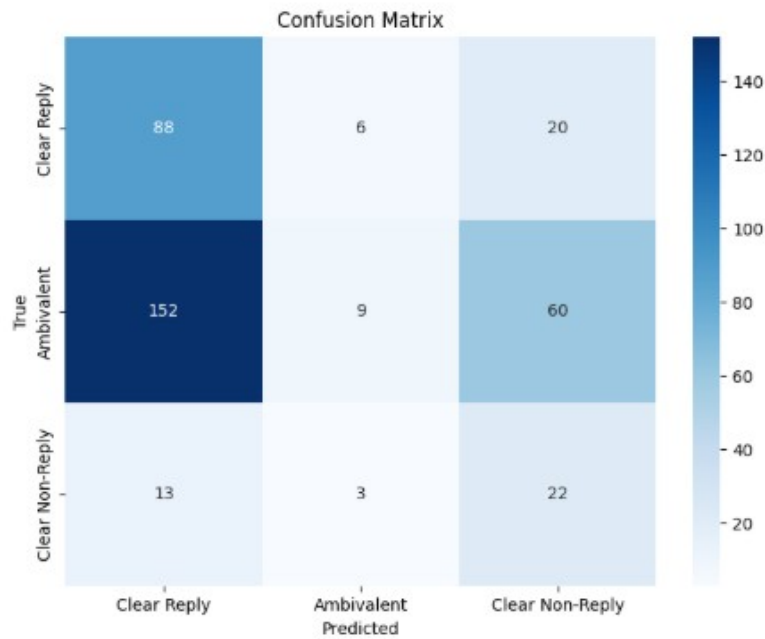


Figure 35: Confusion Matrix

The confusion matrix shows a strong collapse toward the Clear Reply class: the Ambivalent recall drops to 0.04, with 152 Ambivalent answers predicted as Clear Reply and 60 as Clear Non-Reply. This indicates that the reasoning step does not help the model recognise the majority class but instead amplifies its Clear-Reply bias. The accuracy-by-answer-length analysis also shows that the model performs much worse on the longest answers compared to the shorter length groups.

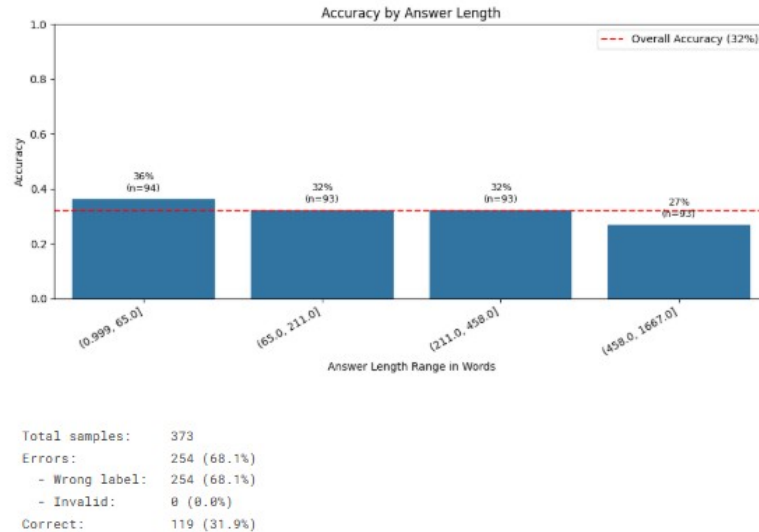


Figure 36: Accuracy by answer length

Overall, CoT does not provide a benefit over the simpler prompting strategies in this experiment. Instead, it appears to increase the complexity of the generation without improving class separation,

Chain-of-Thought with Few shot Chain-of-Thought with few-shot prompting was also tested in order to examine whether providing examples with both reasoning and labels could improve the model’s behaviour. In this setting, each demonstration contains a short reasoning step followed by the gold label, using the same Reasoning: / Label: format expected from the model.

This experiment was considered as an additional exploratory setting, but it was computationally expensive and significantly slower than the previous methods. For this reason, it was not used in the final submission run. Moreover, the few-shot examples were selected randomly rather than with semantic kNN, so that the same demonstrations could be reused across instances and the experiment could be processed in batches.

The results show that this method does not improve performance. The model achieves macro-F1 around 0.30 and weighted-F1 around 0.25, while the confusion matrix shows that the collapse toward the Clear Reply class continues. Therefore, adding few-shot demonstrations with reasoning does not solve the class-separation problem for the smallest model. Instead, it increases the computational cost without producing a clear improvement in performance.

Self-Consistency. Self-Consistency samples the Chain-of-Thought prompt multiple times for the same input and selects the final prediction through majority voting over the valid labels. This experiment was tested to examine whether aggregating several reasoning paths could make the predictions more stable and reduce the bias observed in plain CoT.

The results are very similar to the plain Chain-of-Thought setting. The model achieves accuracy 0.32, macro-F1 0.30, and weighted-F1 0.26, showing no clear improvement. The confusion matrix indicates that the model continues to collapse to-

ward the Clear Reply class . Therefore, the majority vote does not correct the systematic bias; instead, it mostly reinforces the same repeated mistake across sampled generations.

Due to its higher computational cost, this method was not used in the final submission run. Overall, Self-Consistency did not provide a practical benefit in this setting, since the additional sampling increased inference time without improving class separation.

6.3.2. Qwen3.5-2B Experiments.

Zero-shot. The same zero-shot baseline prompt is applied to the larger Qwen3.5-2B model in order to isolate the effect of model size under identical instructions. The model reaches an accuracy of 0.338, a macro-F1 of 0.306, and a weighted-F1 of 0.193, which is substantially lower than the Qwen3.5-0.8B baseline, where the macro-F1 was about 0.45.

Despite being larger, the Qwen3.5-2B model does not improve performance in the zero-shot setting. The confusion matrix shows that it collapses almost entirely toward the Clear Reply class. As a result, many Ambivalent and Clear Non-Reply instances are incorrectly classified as Clear Reply. This leads to poor class separation and explains the lower macro-F1 score.

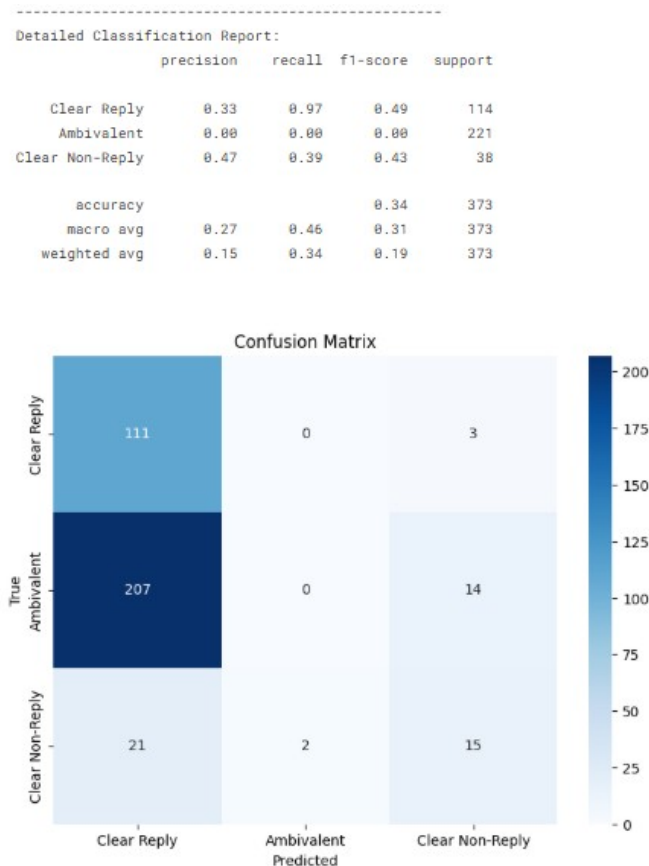


Figure 37: Confusion Matrix Zero Shot for Qwen3.5-2B

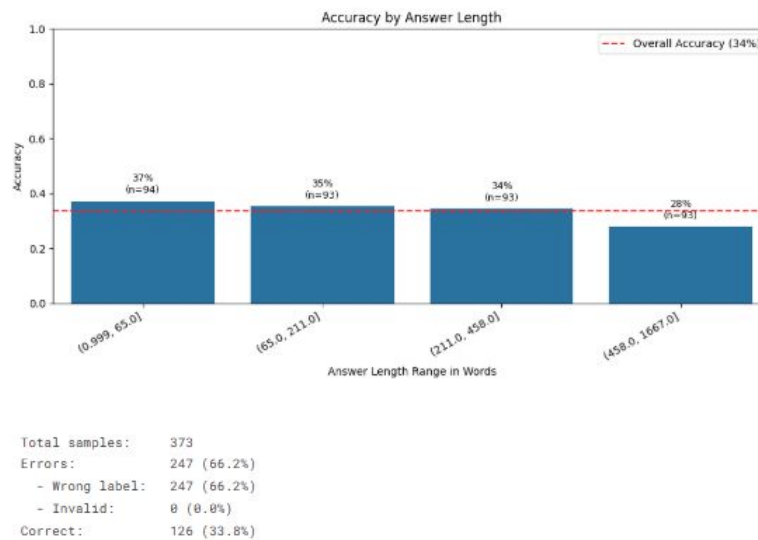


Figure 38: Accuracy by answer length

Few-shot (semantic kNN, $K=3$). The best few-shot configuration in terms of weighted-F1 on the smaller model, namely semantic kNN few-shot prompting with $K=3$ unbalanced examples, was then applied to the Qwen3.5-2B model. Under this setting, increasing the model size improved balanced performance: macro-F1 rose to 0.470, while accuracy and weighted-F1 reached 0.469 and 0.446, respectively.

This result shows that semantically selected demonstrations helped the 2B model substantially. In zero-shot, the same model collapsed toward Clear Reply, reaching only 0.306 macro-F1. With semantic kNN few-shot prompting, this bias was reduced and the predictions became more balanced across the three classes, increasing macro-F1 to 0.470.

This behaviour differs from Qwen3.5-0.8B. For the smaller model, the zero-shot baseline already separated the classes relatively well in terms of macro-F1, while few-shot mainly improved accuracy and weighted-F1. For Qwen3.5-2B, however, few-shot was more important because it corrected a strong zero-shot bias. As a result, the 2B few-shot model achieved a higher macro-F1 than the 0.8B few-shot model, although the 0.8B model remained stronger in weighted-F1, which is the Kaggle evaluation metric.

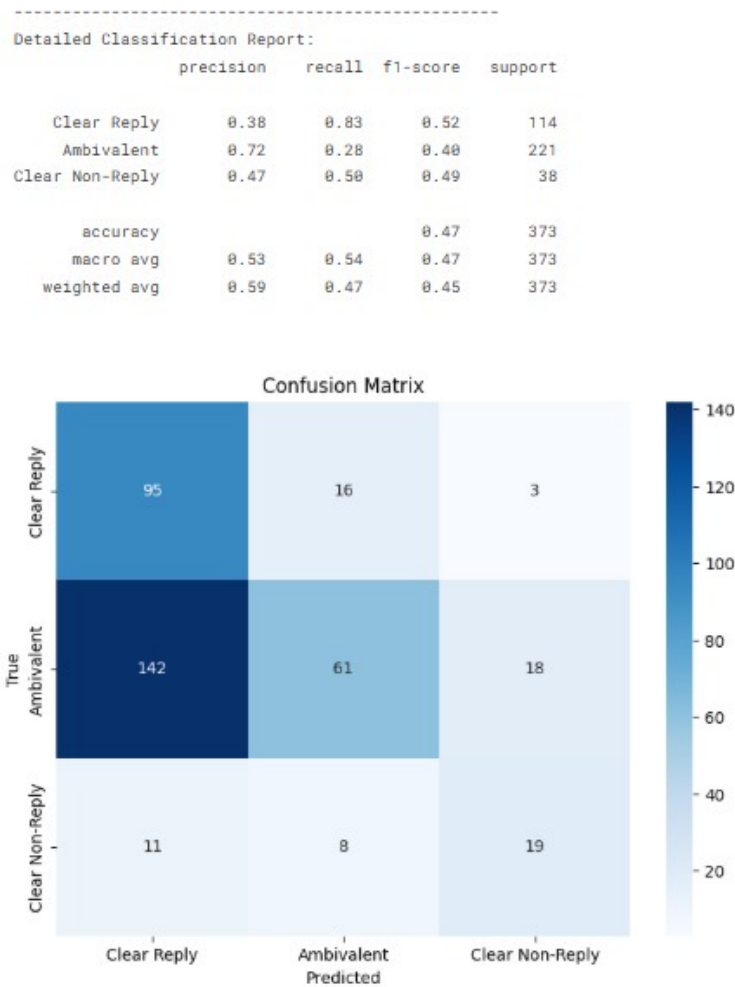


Figure 39: Confusion matrix for Qwen3.5-2B with semantic kNN few-shot prompting.

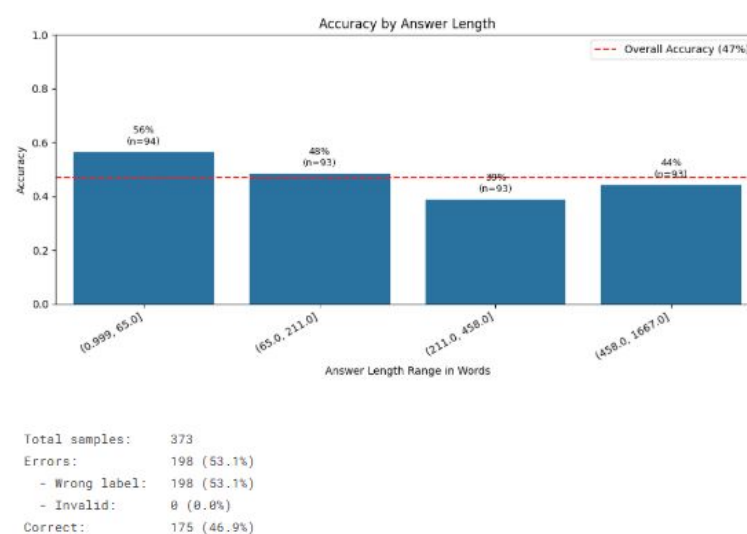


Figure 40: Accuracy by answer length

The answer-length analysis does not reveal substantial performance differences

across length groups, although a small drop is observed for longer answers.

Chain-of-Thought. Chain-of-Thought was also tested at this model size, but it did not yield any notable improvement over the smaller model. The model classified most instances as Clear Reply and Clear Non-Reply, while predicting only very few examples as Ambivalent. As indicative values, it obtained a macro-F1 of 0.32 and a weighted-F1 of 0.29.

This result confirms the pattern already observed with the smaller model: Chain-of-Thought prompting does not perform well for this task. Although the method encourages the model to generate an intermediate reasoning step, this reasoning does not help the model distinguish whether the answer actually addresses the specific sub-question. Instead, the model continues to show strong label bias and poor class separation. Since CoT is also much slower, especially on larger models, this setting was not included in the main final run.

⚡subsectionQwen3.5-4B Experiments

Zero-shot (baseline). The baseline prompt is applied to the largest Qwen model, Qwen3.5-4B, giving accuracy 0.370, macro-F1 0.334, and weighted-F1 0.265.

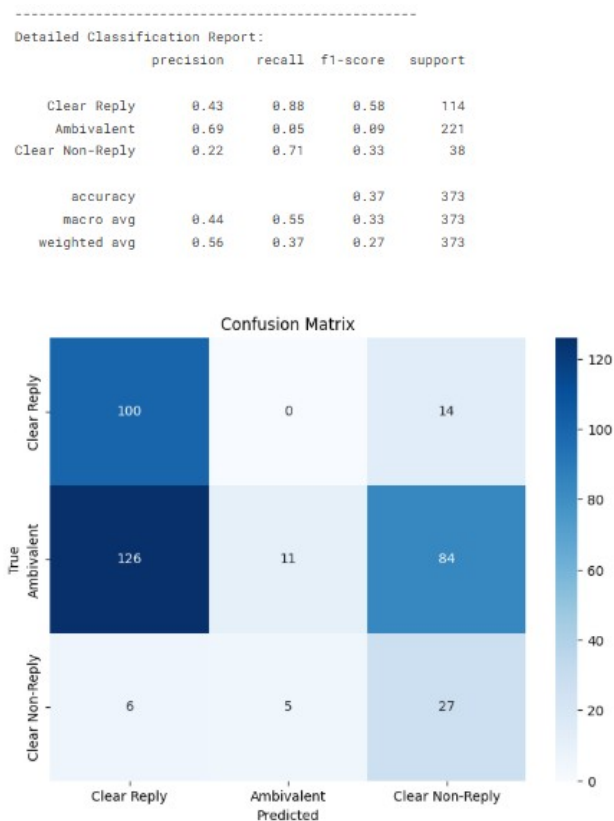


Figure 41: Confusion Matrix Zero Shot for Qwen3.5-4B

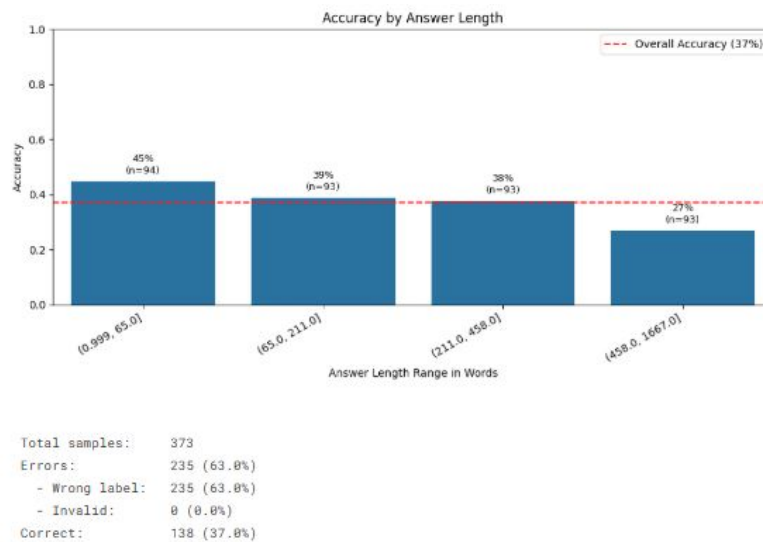


Figure 42: Accuracy by answer length

Like the 2B model, Qwen3.5-4B is also biased in the zero-shot setting, especially toward Clear Reply. However, the collapse is slightly less extreme than in the 2B model, since the 4B model distributes some predictions across the other classes as well. This leads to a small improvement in macro-F1 compared with Qwen3.5-2B, but the score still remains clearly below the Qwen3.5-0.8B zero-shot baseline.

Few-shot (semantic kNN, K= 3). The semantic kNN few-shot configuration with K= 3 unbalanced examples was also applied to the Qwen3.5-4B model. With this setting, the 4B model achieved the best macro-F1 of all tested systems, reaching 0.479, with an accuracy of 0.496 and a weighted-F1 of 0.490.

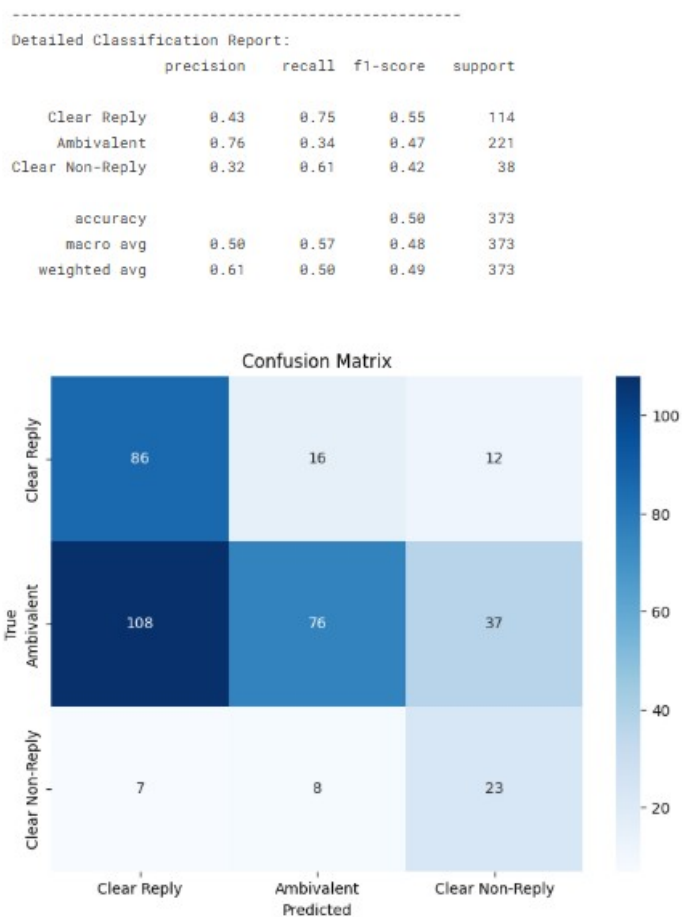


Figure 43: Confusion matrix for Qwen3.5-4B with semantic kNN few-shot prompting.

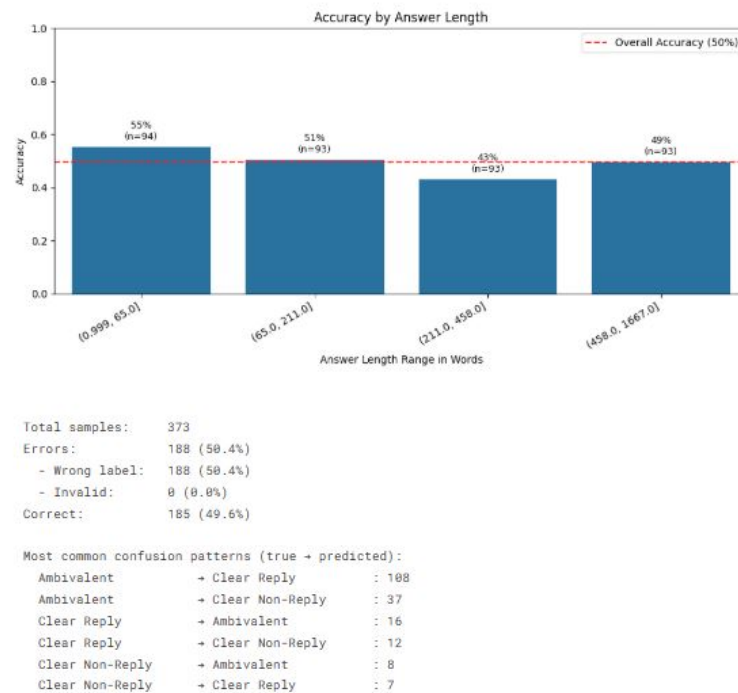


Figure 44: Accuracy by answer length

This result shows the clearest positive effect of model size in the few-shot setting. While the 4B model was still biased in zero-shot, the semantically selected demonstrations allowed it to use its larger capacity more effectively. Compared with Qwen3.5-2B few-shot, the improvement is small but consistent, with macro-F1 increasing from 0.470 to 0.479. This is the best macro-F1 obtained in the experiments, indicating the strongest balanced performance across the three classes.

However, this advantage appears mainly in macro-F1. The 4B model does not achieve the best accuracy or weighted-F1, since Qwen3.5-0.8B with semantic kNN few-shot remains stronger on those metrics. Therefore, Qwen3.5-4B is preferable when balanced class performance is prioritised, whereas the smaller 0.8B model is more suitable when weighted-F1 and runtime efficiency are the main criteria.

Chain-of-Thought. Chain-of-Thought was also tested on the Qwen3.5-4B model, but it again did not perform well. As indicative values, it reached a macro-F1 of 0.34 and a weighted-F1 of 0.30, which are clearly lower than the few-shot result of the same model.

This confirms the pattern observed in the previous models: adding an explicit reasoning step does not improve performance for this task.

6.3.3. Gemma 3 1B.

Zero-shot. To test whether the trends observed in the Qwen family also appear in a different model family, the small Gemma 3 1B model is evaluated under the same baseline zero-shot prompt. It reaches an accuracy of 0.290, a macro-F1 of 0.264, and a weighted-F1 of 0.176.

Detailed Classification Report:

	precision	recall	f1-score	support
Clear Reply	0.36	0.66	0.47	114
Ambivalent	0.00	0.00	0.00	221
Clear Non-Reply	0.20	0.87	0.32	38
accuracy			0.29	373
macro avg	0.19	0.51	0.26	373
weighted avg	0.13	0.29	0.18	373

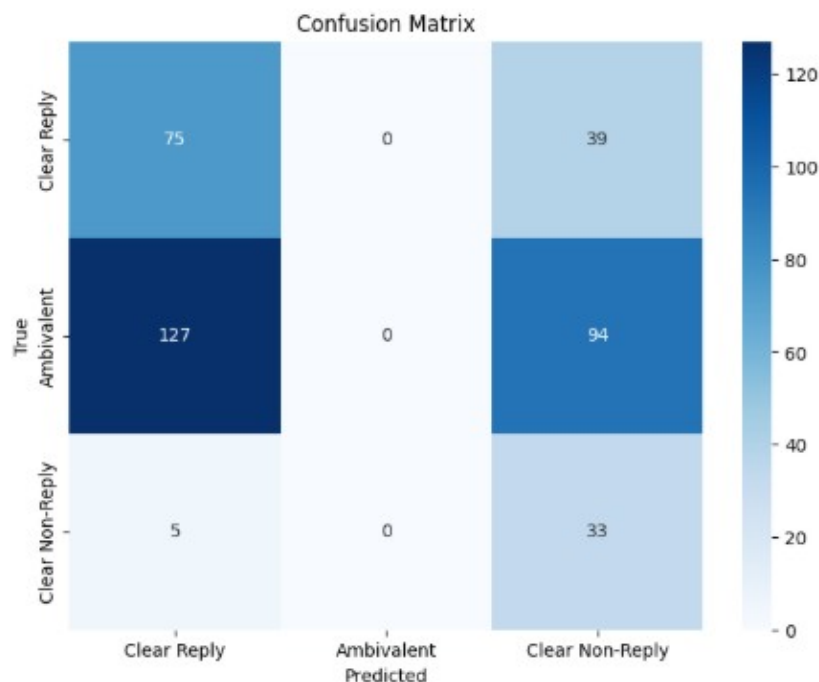


Figure 45: Confusion matrix for Gemma 3 1B with zero-shot prompting.

Unlike Qwen3.5-0.8B, which performed relatively well in zero-shot, Gemma 3 1B struggles under the same prompting setting. This suggests that the behaviour of small instruction-tuned models is not determined only by parameter size, but also by the model family and its instruction-following behaviour.

Few-shot (semantic kNN, K= 3). Semantic kNN few-shot prompting also improved Gemma 3 1B substantially. In the zero-shot setting, the model performed weakly, reaching only 0.264 macro-F1 and 0.176 weighted-F1. After adding semantically selected demonstrations, its performance increased to 0.42 macro-F1 and 0.46 weighted-F1. This shows that, as in the Qwen3.5-2B case, relevant few-shot examples helped reduce the weak zero-shot behaviour and improved class separation.

However, Gemma 3 1B still did not become the strongest configuration. Its few-shot performance was close to Qwen3.5-0.8B in terms of macro-F1, but it remained weaker in weighted-F1. It also remained below Qwen3.5-2B few-shot in macro-F1. Therefore, the result is useful as evidence that semantic kNN few-shot prompting helps across model families, but it was treated as an indicative cross-family experiment and was

not included in the main final run due to time constraints.

6.3.4. Gemma 3 4B.

Zero-shot. The larger Gemma 3 4B model provides a second cross-family comparison point at a scale comparable to Qwen3.5-4B. Under the same zero-shot baseline prompt, it achieved an accuracy of 0.539, a macro-F1 of 0.417, and a weighted-F1 of 0.527.

Detailed Classification Report:

	precision	recall	f1-score	support
Clear Reply	0.44	0.39	0.41	114
Ambivalent	0.61	0.68	0.64	221
Clear Non-Reply	0.25	0.16	0.19	38
accuracy			0.54	373
macro avg	0.43	0.41	0.42	373
weighted avg	0.52	0.54	0.53	373

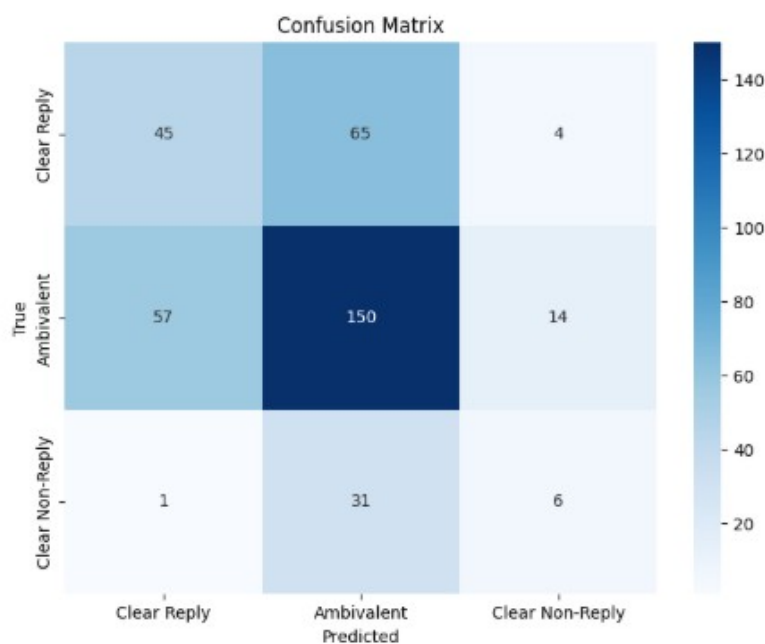


Figure 46: Confusion matrix for Gemma 3 4B with zero-shot prompting.

This is a strong zero-shot result, especially compared with the larger Qwen3.5 models. While Qwen3.5-2B and Qwen3.5-4B showed a strong zero-shot bias toward Clear Reply, Gemma 3 4B remained more stable and achieved both a good macro-F1 and a good weighted-F1. This suggests that the observed behaviour is not determined only by model size, but also by the model family and its instruction-following behaviour.

Few-shot (semantic kNN, K= 3). Semantic kNN few-shot prompting also improved Gemma 3 4B, reaching a macro-F1 of 0.47 and a weighted-F1 of 0.53. This makes it one

of the strongest overall configurations, because it performs well on both balanced performance (macro-F1) and frequency-weighted performance (weighted-F1). In contrast, some other models performed well on one metric but not on the other.

The result is comparable to the best Qwen3.5 few-shot configurations. In terms of macro-F1, Gemma 3 4B is close to Qwen3.5-4B few-shot, which achieved the best macro-F1 overall (0.479). In terms of weighted-F1, it is close to the Qwen3.5-0.8B few-shot model, which achieved the best weighted-F1 (0.538). Therefore, Gemma 3 4B can be considered the most balanced cross-family result, even though the main comparison and final model selection were based on the Qwen3.5 experiments.

In this setting, the model also produced two `Invalid` predictions, but the invalid-output rate remained very low. This configuration was treated as an indicative cross-family experiment and was not included in the main final run due to time constraints.

6.4. Overall Results

Prompt design. The prompt-design experiments were conducted in order to examine how the model’s performance changes when the prompt formulation is modified. The results are reported in Table 4. Several clear patterns emerge. First, the baseline zero-shot prompt is the strongest zero-shot variant, reaching a macro-F1 of 0.447. This shows that, for Qwen3.5-0.8B, a short but well-structured instruction was more effective than adding more detailed guidance.

The detailed prompt, which added per-label definitions and the heuristic “longer answers are Ambivalent, shorter ones are Clear Non-Reply”, reduced macro-F1 to 0.266. This suggests that the additional rules did not help the model make finer distinctions between the classes. Instead, the length-based heuristic appears to have introduced an unintended bias, which pushed the model toward incorrect label choices.

The simplified prompt and the only-user-message variant also performed worse than the baseline. This shows that the small Qwen model is sensitive not only to the content of the instruction, but also to how the instruction is placed in the chat template. In particular, when the instruction was moved from the system message into the user message, performance dropped, suggesting that Qwen3.5-0.8B benefits from receiving the classifier instruction as a separate system role. This is also important for the cross-family comparison, because Gemma 3 does not preserve the system message in the same way and instead merges the system content into the user turn. Therefore, prompt structure and chat-template behaviour should be considered part of the experimental setup, since different model families may respond differently to the same instruction content.

Configuration	Acc.	F1-Macro	F1-W	Prec.	Rec.	Inv.%
Zero-shot (baseline)	0.515	0.447	0.520	0.446	0.456	0.0
Zero-shot (simplified)	0.357	0.314	0.321	0.355	0.394	0.0
Zero-shot (detailed)	0.306	0.266	0.221	0.328	0.406	0.0
Zero-shot (only user msg)	0.389	0.300	0.372	0.322	0.342	0.0
Few-shot (random)	0.563	0.274	0.459	0.399	0.325	0.0
Few-shot kNN (K=3, bal.)	0.536	0.382	0.513	0.400	0.388	0.3
Few-shot kNN (K=3, unbal.)	0.582	0.416	0.538	0.504	0.408	0.5
Few-shot kNN (K=6, bal.) [†]	0.540	0.430	0.530	–	–	0.0
Chain-of-Thought	0.319	0.290	0.223	0.355	0.464	0.0
CoT + Few-shot [†]	0.313	0.300	0.250	–	–	–
Self-Consistency [†]	0.320	0.300	0.260	–	–	–

Table 4: Qwen3.5-0.8B: prompt-design and prompting-method variants on exp_df. Best values per column are shown in bold.

[†] Indicative experiment: this configuration was evaluated and discussed for analysis, but was not kept as part of the main final experimental run because it did not improve the main results and/or required higher runtime.

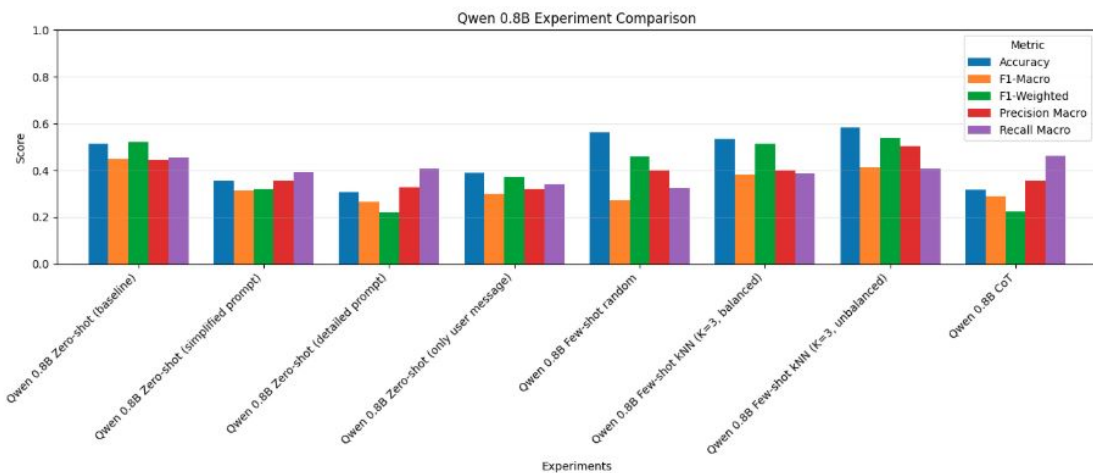


Figure 47:

Prompt method comparison on Qwen3.5-0.8B. Among the prompting methods, few-shot demonstrations improved the model’s performance in terms of accuracy and weighted-F1, but their effect on macro-F1 was mixed. Random few-shot selection achieved high accuracy (0.563), but it also produced the lowest macro-F1 among the 0.8B configurations (0.274). This happened because the model collapsed toward the majority class: Ambivalent recall reached 0.92, while Clear Reply recall dropped to 0.05 and Clear Non-Reply was not predicted at all (F1 = 0.00). Therefore, the improvement in accu-

racy did not reflect better balanced classification, but mainly better performance on the dominant class.

Semantic kNN selection corrected part of this problem by selecting demonstrations that were more relevant to each test instance. The unbalanced $K=3$ variant was the strongest 0.8B configuration when weighted-F1 was prioritised, reaching an accuracy of 0.582 and a weighted-F1 of 0.538. However, its macro-F1 was 0.416, which remained lower than the zero-shot baseline macro-F1 of 0.447. This shows that semantic few-shot prompting helped the model overall under the weighted-F1 metric, but did not fully improve balanced performance across all three classes.

The balanced $K=3$ variant achieved a lower macro-F1 (0.382), suggesting that forcing one example per class was not enough on its own. Increasing the number of demonstrations to $K=6$ improved macro-F1 slightly, reaching 0.430, because the prompt still preserved class coverage while adding more semantically similar examples. However, $K=6$ did not improve weighted-F1 compared with the smaller unbalanced $K=3$ setting. Therefore, $K=6$ was useful for balanced class performance, but not for the official weighted-F1 criterion.

Finally, Chain-of-Thought prompting was detrimental for this model, reaching only 0.290 macro-F1. Instead of helping the model reason more carefully about the relation between the question and the answer, the reasoning step amplified the Clear Reply bias. The same pattern was also observed in the additional CoT variants that were tested, such as CoT with few-shot demonstrations and Self-Consistency. Although these variants were designed to make the reasoning more stable or better guided, they did not improve the final performance and mainly increased the computational cost.

Overall, the baseline zero-shot prompt remained the strongest 0.8B configuration in terms of macro-F1, while the unbalanced semantic kNN few-shot setting with $K=3$ was the strongest when weighted-F1 and accuracy were prioritised.

Effect of model size. Table 5 compares the three Qwen3.5 models under the main settings used for the model-size analysis: the baseline zero-shot prompt and the semantic kNN few-shot configuration with $K=3$. Chain-of-Thought is also included as an indicative setting, since it was tested for analysis but was not kept as part of the main final run due to its higher computational cost and weaker performance. The results show different trends depending on the prompting method.

In zero-shot, increasing the model size did not improve performance. The smallest model achieved the highest zero-shot macro-F1 (0.447) and weighted-F1 (0.520), while the 2B and 4B models performed worse. The main reason is that the larger zero-shot models developed a strong bias toward Clear Reply. As a result, they misclassified many Ambivalent examples as direct answers, which reduced both their macro-F1 and weighted-F1 scores.

In semantic kNN few-shot prompting, the trend was different. The larger models benefited more from semantically selected demonstrations. Macro-F1 increased from 0.416 for Qwen3.5-0.8B to 0.470 for Qwen3.5-2B and 0.479 for Qwen3.5-4B. This shows that model size helped mainly when the model was guided by relevant examples. The demonstrations reduced the zero-shot label bias and helped the larger models distribute predictions more effectively across the three classes.

However, the 0.8B model still achieved the highest accuracy and weighted-F1 in the few-shot setting, with accuracy 0.582 and weighted-F1 0.538. This makes it the most practical choice when the official weighted-F1 metric and inference cost are prioritised.

In contrast, Qwen3.5-4B is preferable when macro-F1 is the main criterion, because it provides the best balanced performance across the three classes.

The Chain-of-Thought results confirm the pattern observed in the individual experiments. CoT did not improve performance at any model size and remained clearly below the semantic few-shot setting. Although the larger models achieved slightly higher indicative macro-F1 scores with CoT than the 0.8B model, the method still suffered from label bias and high runtime. Therefore, CoT was useful mainly as an exploratory comparison, but it was not competitive with semantic kNN few-shot prompting.

Model	Method	Acc.	F1-Macro	F1-W	Prec.	Rec.
Qwen3.5-0.8B	Zero-shot	0.515	0.447	0.520	0.446	0.456
Qwen3.5-2B	Zero-shot	0.338	0.306	0.193	0.265	0.456
Qwen3.5-4B	Zero-shot	0.370	0.334	0.265	0.445	0.546
Qwen3.5-0.8B	Few-shot (kNN K=3)	0.582	0.416	0.538	0.504	0.408
Qwen3.5-2B	Few-shot (kNN K=3)	0.469	0.470	0.446	0.525	0.537
Qwen3.5-4B	Few-shot (kNN K=3)	0.496	0.479	0.490	0.502	0.568
Qwen3.5-0.8B	Chain-of-Thought [†]	0.319	0.290	0.223	0.355	0.464
Qwen3.5-2B	Chain-of-Thought [†]	–	0.320	0.290	–	–
Qwen3.5-4B	Chain-of-Thought [†]	–	0.340	0.300	–	–

Table 5: Effect of model size in the Qwen3.5 family under zero-shot, semantic kNN few-shot prompting, and indicative Chain-of-Thought runs on exp_df. Best values within the zero-shot and few-shot blocks are shown in bold.

[†] Indicative experiment: Chain-of-Thought was tested for analysis, but was not included in the main final experimental run because it was slower and did not improve performance.

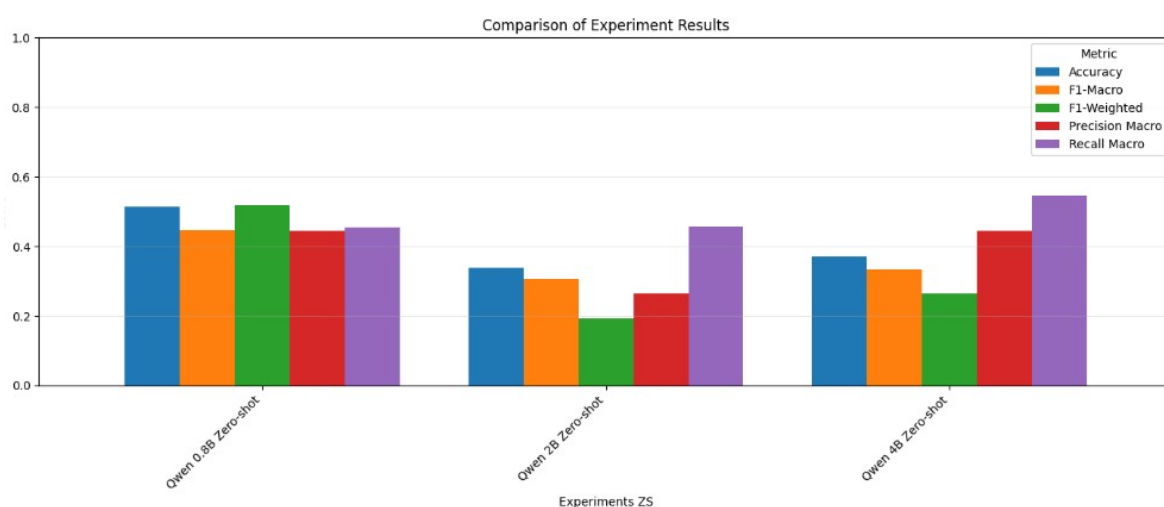


Figure 48:

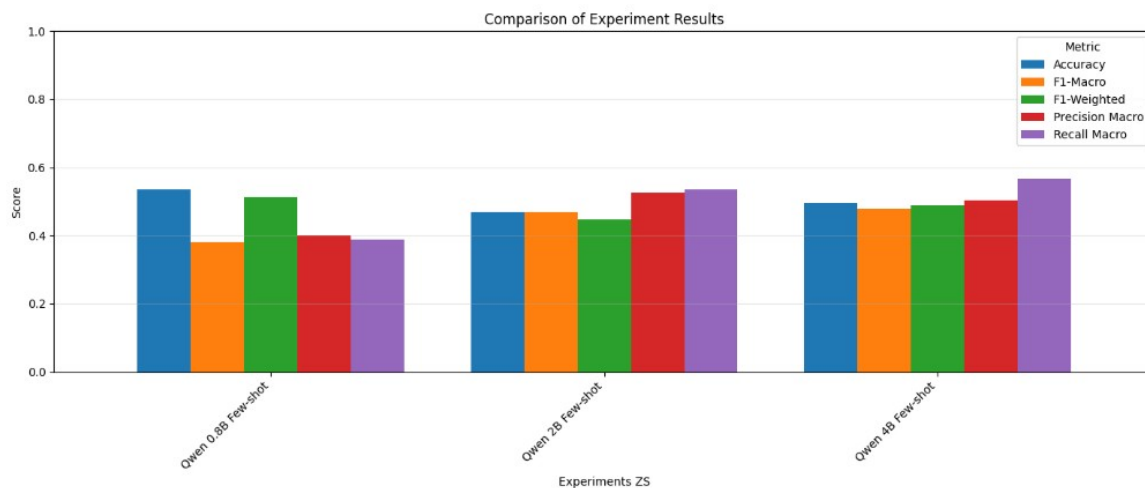


Figure 49:

Comparison with the Gemma 3 family. The Gemma 3 experiments were included in order to examine whether the trends observed in the Qwen3.5 family also appear in a different model family. The results show that the behaviour is not identical across architectures. In zero-shot prompting, Gemma 3 1B performed weakly, with macro-F1 0.264 and weighted-F1 0.176, showing that small instruction-tuned models can struggle when they receive only the task instruction. In contrast, Gemma 3 4B achieved a much stronger zero-shot result, with macro-F1 0.417 and weighted-F1 0.527, which is substantially better than the zero-shot results of the larger Qwen3.5 models.

Few-shot prompting also improved the Gemma models. Gemma 3 1B reached macro-F1 0.42 and weighted-F1 0.46, while Gemma 3 4B reached macro-F1 0.47 and weighted-F1 0.53. These results are comparable to the best Qwen3.5 few-shot configurations, showing that semantic few-shot prompting is useful beyond a single model family.

In particular, Gemma 3 4B achieved strong performance on both macro-F1 and weighted-F1, making it one of the most balanced configurations overall. However, the Gemma few-shot runs were treated as indicative cross-family experiments and were not kept as part of the main final experimental run, mainly due to time constraints.

Overall, the Gemma results confirm that model family matters. Unlike the Qwen3.5 models, where larger zero-shot models became strongly biased toward Clear Reply, Gemma 3 4B performed strongly even in zero-shot. This suggests that the observed failure modes are not caused only by model size, but also by model-specific instruction-following behaviour and calibration. Nevertheless, across both families, semantically selected few-shot demonstrations remained one of the most effective ways to improve performance.

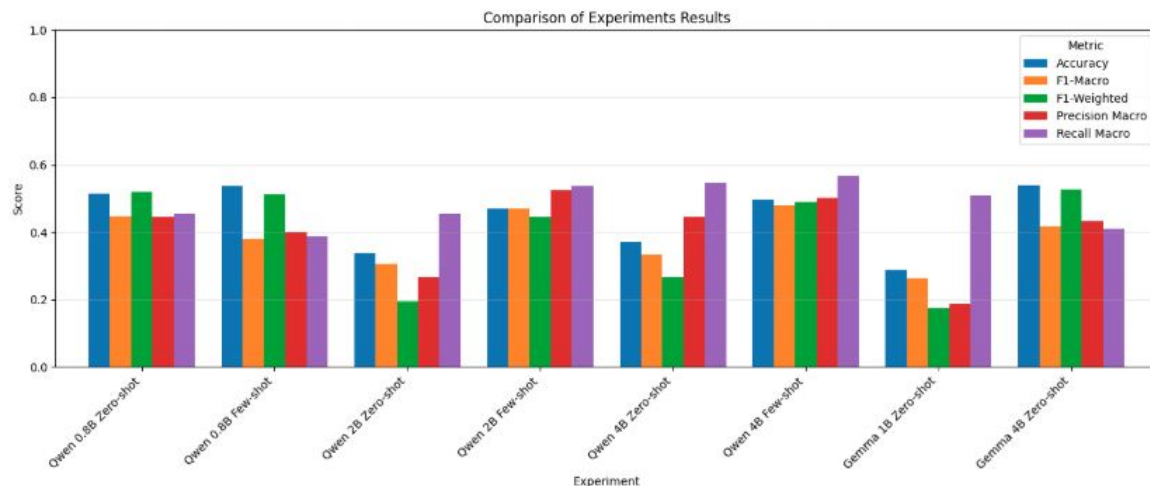


Figure 50:

Performance across answer-length subgroups. The answer-length analysis shows that length is related to model performance, but this effect is closely connected to the class distribution. In the dataset, Ambivalent and Clear Reply answers tend to be longer, while Clear Non-Reply answers are usually much shorter. Therefore, lower performance on short answers may partly reflect the difficulty of the Clear Non-Reply class rather than a pure length effect.

Short answers were generally harder for the models, because many of them belong to the minority Clear Non-Reply class. The models often confused these cases with Ambivalent, especially when the answer was short but still topically related to the question. For medium and longer answers, performance was more stable, but the main confusion between Ambivalent and Clear Reply remained.

Overall, answer length provides some useful signal, but it is not sufficient to solve the task. The models still need to determine whether the answer addresses the specific sub-question directly, partially, or not at all.

7. Error Analysis

Format errors. Format errors were rare across the experiments. The invalid-output rate remained very low, with at most two invalid predictions out of 373 instances. The few invalid outputs were usually very short responses such as “Yes” or “No”, mainly for Clear Non-Reply examples.

This shows that the output format was generally well controlled. Since the generation length was restricted and the prompt explicitly required the model to output only one label, most models produced valid and easily parsable labels. Therefore, the main source of error was not output formatting, but the semantic difficulty of distinguishing between Clear Reply, Ambivalent, and Clear Non-Reply.

Class-separation errors. The result analysis shows that the main difficulty across the experiments is the separation of the three clarity classes. The models often fail to distinguish between a direct answer, a partial or unclear answer, and a complete non-reply.

This problem appears in different forms depending on the model and the prompting method.

In several configurations, the model collapses toward a single label. For example, random few-shot prompting on Qwen3.5-0.8B over-predicts *Ambivalent*, while the larger Qwen3.5 zero-shot models over-predict *Clear Reply*. This shows that the models are sensitive to prompt design, model size, and demonstration selection. Instead of consistently separating the three classes, they often follow a biased decision pattern.

The most frequent semantic confusion is *Ambivalent* $\rightarrow$ *Clear Reply*. This means that the models often treat a fluent and topic-relevant answer as a direct reply, even when the answer only partially or unclearly addresses the specific sub-question.

At the same time, *Clear Non-Reply* remains the most difficult class to recover reliably. This is partly because it is the minority class and partly because many non-replies are short, which makes them easy to confuse with short or partial *Ambivalent* answers. In some configurations, such as random few-shot selection, *Clear Non-Reply* is not predicted at all. Even its best individual F1-score remains 0.49, which is lower than the best individual F1-scores achieved by the other classes. This shows that the models struggle to identify complete non-replies consistently.

Therefore, the task requires more than detecting whether the answer is generally related to the topic; it requires checking whether the exact point of the question has been answered.

Overall, stronger configurations tend to be those that avoid collapsing into a single label and achieve more balanced individual F1-scores across the three classes. Semantic kNN few-shot prompting helps in this direction, especially for the larger models, because the selected examples guide the model toward a more balanced decision boundary. However, even the best models still confuse *Ambivalent* with *Clear Reply*, which remains the central error pattern of the task.

Model and family-specific failure patterns. The error patterns also suggest that failure modes depend not only on model size, but also on the model family. Within the Qwen3.5 family, the smaller Qwen3.5-0.8B model appears more sensitive to prompt wording, with errors distributed more broadly across classes. In contrast, the larger Qwen3.5 models show a stronger label bias in the zero-shot setting, especially toward *Clear Reply*. This explains why the larger Qwen models perform worse than the smaller model in zero-shot, despite having more parameters.

However, when semantically selected few-shot examples are added, this bias is reduced. As a result, the larger Qwen models benefit more from the demonstrations and achieve better macro-F1 scores in the few-shot setting. This shows that model size helps mainly when the model is guided by relevant examples, rather than in the zero-shot setting.

The Gemma results show that this behaviour is not universal across model families. Gemma 3 4B performs strongly even in zero-shot, achieving both good macro-F1 and weighted-F1. Therefore, the collapse observed in the larger Qwen zero-shot models should not be interpreted as a general effect of scale alone. It also depends on model-specific instruction-following behaviour, calibration, and how each model family responds to the prompt format.

Different prompting methods fail for different reasons. The different prompting strategies produced different failure modes. The detailed zero-shot prompt did not improve performance, even though it included label definitions and heuristic rules. Instead, the additional rules appear to have introduced an unintended bias, especially through the length-based heuristic, which distorted the model’s decision boundary.

Few-shot prompting behaved differently. Random few-shot selection was not reliable, because the selected examples were not necessarily relevant to the test instance and led the model to over-predict the majority class. In contrast, semantic kNN selection was more effective because it selected demonstrations that were closer to the input. This shows that few-shot prompting depends not only on the number of examples, but also on their relevance.

The effect of few-shot prompting was especially important for the larger Qwen3.5 models. In zero-shot, these models showed a strong bias toward Clear Reply, but semantically selected demonstrations helped reduce this collapse and produced more balanced predictions. Therefore, examples did not simply improve performance in general; they changed the type of errors made by the larger models.

The comparison between balanced and unbalanced semantic kNN also shows a trade-off. The unbalanced $K=3$ setting achieved the best weighted-F1 for Qwen3.5-0.8B, while the balanced $K=6$ setting improved macro-F1 among the few-shot variants by preserving class coverage and adding more semantically similar examples. However, this improvement was not enough to outperform the zero-shot baseline in macro-F1, and the $K=6$ setting required more runtime. Therefore, the best few-shot configuration depends on whether weighted-F1, macro-F1, or computational cost is prioritised.

Finally, Chain-of-Thought prompting did not lead to clear improvements. Although it encouraged the model to generate an explanation before the final label, the reasoning was not always useful for the actual decision. In several cases, the model produced plausible reasoning while still assigning an incorrect label. The same pattern was observed in all CoT-based variants that were tested, including CoT with few-shot demonstrations and Self-Consistency. These variants also failed to improve class separation and mainly increased the computational cost. Therefore, simply asking the model to reason step by step was not sufficient; future reasoning prompts may need to focus more explicitly on whether the answer addresses the specific sub-question.

paragraphWhat an improved system would need.

Overall, the error analysis suggests that better performance would require a system that focuses more explicitly on the relation between the answer and the specific sub-question. This difficulty is partly related to the structure of the dataset. Each instance pairs a sub-question with the full interview answer, but the exact part of the answer that is relevant to that sub-question is not explicitly isolated. As a result, the model has to solve two problems at the same time: first, it must identify which part of the answer is relevant, and then it must decide whether that part answers the sub-question clearly, partially, or not at all.

A useful extension of the dataset would therefore be to isolate or annotate the answer spans that correspond to each sub-question. This would make the relevant evidence clearer and could help the model focus on the correct part of the response before assigning the final clarity label. Another useful improvement would be a more balanced dataset, especially with more examples of Clear Non-Reply, since this is the smallest and most difficult class to recover reliably.

Finally, future systems could benefit from clearer decision rules for the three abstract label meanings. The distinction between a clear answer, an ambiguous answer, and a non-reply is difficult because it depends on the whole meaning of the question–answer pair, not on individual words. In many cases, even human readers may find it difficult to decide whether an answer is fully answered, only partially answered, or not answered at all. More explicit rules, together with better and more representative examples, could help the model apply these label definitions more consistently.

8. Conclusion and model selection

For the final Kaggle submission, **Qwen3.5-0.8B with semantic kNN few-shot prompting (K= 3, unbalanced)** was selected. On `exp_df`, this configuration achieved the best accuracy and weighted-F1 among the practical submission candidates, with an accuracy of 0.582 and a weighted-F1 of 0.538. Since the Kaggle leaderboard uses weighted-F1 and the 0.8B model is much cheaper to run on the full test set, it was the most appropriate choice for the final `submission.csv`.

If macro-F1 is prioritised instead, the best Qwen configuration is Qwen3.5-4B with semantic kNN few-shot prompting, which reached a macro-F1 of 0.479. Alternatively, a model from a different family could also be considered: Gemma 3 4B produced strong and balanced results, reaching macro-F1 0.47 and weighted-F1 0.53. This makes it one of the strongest overall configurations, although it was treated as an indicative cross-family experiment rather than the main submission pipeline.

Overall, the final model choice depends on the evaluation objective: the Qwen3.5-0.8B few-shot model is preferable for the official weighted-F1 submission and runtime efficiency, while Qwen3.5-4B or Gemma 3 4B are stronger options when balanced class performance is the main priority.

9. Comparison with the other projects

Compared with the previous projects, the prompting approach used in this work offers a different trade-off between performance, flexibility, and computational cost. In HW1, the task was approached with traditional vector-based representations, mainly TF-IDF and Word2Vec, combined with classical classifiers. These methods were useful as baselines, but their performance depended strongly on preprocessing and feature-engineering choices. The most stable HW1 result was obtained with Word2Vec, reaching an F1-score of approximately 49%, but these representations remained limited because they could not fully capture the semantic relation between the question and the answer.

In HW2, fine-tuned encoder-only Transformer models achieved the strongest results overall. The best model reached a macro-F1 of about 0.60, which is higher than the best prompting result in the current work. This is expected, because the HW2 models were trained directly on the labelled dataset and could learn the task from many examples. The improvement also shows the advantage of contextual Transformer representations over traditional vector-based methods. For this task, the label depends on the relation between the question and the answer, not only on the words that appear in the text.

In contrast, the models in the current project were not fine-tuned. They relied only on prompting, instructions, and a small number of demonstrations. Despite this limi-

tation, the best prompting setup, Qwen3.5-4B with semantic kNN few-shot prompting, reached a macro-F1 of 0.479. This is lower than the fine-tuned Transformer result from HW2, but it is still notable because no model parameters were updated. Therefore, prompting can be considered a flexible low-resource alternative when fine-tuning is not possible, when labelled data is limited, or when a lighter experimental setup is preferred.

Overall, fine-tuning remains the strongest approach for this classification task, while prompting provides a more flexible but less accurate solution. Traditional vector-based methods are useful as simple baselines, but they are limited by their reliance on surface-level lexical information. Prompting-based LLMs, on the other hand, require no additional training and can still reach competitive results, especially when semantically relevant demonstrations are selected.